



Evaluación de una migración de contenedores individuales hacia contenedores orquestados utilizando como facilitador principal Large Language Model (LLM)

Felipe Sáez Vergara

`felipe.saez@usm.cl`

Resumen: Esta tesina explora el potencial de los Large Language Models (LLM) públicos, como ChatGPT, para facilitar la adopción de tecnologías complejas como Kubernetes en entornos empresariales. Se investiga cómo los LLM pueden ayudar a superar la curva de aprendizaje y simplificar la migración de contenedores en Docker a Kubernetes mediante el uso de instrucciones y *prompts* específicos. Se desarrolla una herramienta personalizada que interactúa con la API de OpenAI para generar configuraciones de Kubernetes a partir de archivos Docker, demostrando la viabilidad de utilizar LLM en tareas de automatización y configuración. Los resultados sugieren que los LLM pueden acelerar la adopción de nuevas tecnologías, pero también resaltan la necesidad de supervisión experta y validación de las soluciones generadas. Se concluye que los LLM, combinados con una interacción adecuada a través de APIs y un conocimiento sólido en *prompt engineering*, ofrecen un enfoque prometedor para simplificar la adopción de tecnologías y mejorar la eficiencia en entornos empresariales.

Palabras Clave: Kubernetes, Docker, Large Language Models (LLM), ChatGPT, adopción de tecnología, migración de contenedores, automatización, *prompt engineering*, inteligencia artificial.

1. Introducción

Actualmente la orquestación de contenedores con Kubernetes es el estándar de facto y en el 2022 aumentó su crecimiento anual en un 127% [1]. Sin embargo, la adopción de esta tecnología, ya sea a través de servicios manejados como Google Kubernetes Engine o Azure Kubernetes Service, es una tarea compleja, lo que puede resultar un obstáculo para empresas que no tienen los recursos técnicos y humanos para realizar la migración. En esta investigación se propuso evaluar una migración desde contenedores individuales ejecutados en Docker, simulando una empresa sin conocimientos sobre Kubernetes que realiza esta migración, lo que permitiría orquestar estos contenedores de una manera organizada y con mayor gestión, pero apoyado en el uso de Large Language Model (LLM) públicos como lo son los chatbots Bard y ChatGPT y también el uso de LLM como servicio a través de API. Debido a la importancia de la Inteligencia Artificial y su crecimiento exponencial para el desarrollo de software —como por ejemplo herramientas como Github Copilot y Ansible Lightspeed—, se decidió investigar acerca de su uso para el caso señalado.

La problemática principal radica en la complejidad de adopción de un orquestador de contenedores como Kubernetes, el cual a la larga permite una mejor gestión en escalabilidad, gobernabilidad y orquestación y, a la vez, en la oportunidad potencial de los modelos de lenguajes grandes (LLM), debido a que éstos aprenden de grandes cantidades de datos, incluyendo las configuraciones referentes a Kubernetes, Docker, entre otros.

El uso de LLM respecto a una migración ofrece una oportunidad para aumentar la automatización, gestionar mejor la documentación, levantamiento de riesgos y seguridad.



1.1 Problema detectado

El problema detectado corresponde a la complejidad en pasar desde contenedores individuales hacia un orquestador de contenedores como Kubernetes.

En primer lugar, existe una curva de aprendizaje respecto al uso de Kubernetes, el cómo operar los contenedores y los distintos objetos que van apareciendo en las configuraciones, como por ejemplo, los objetos de *Service*, *Deployment*, entre otros. También existen diferencias referente al uso de las redes, la configuración interna de los DNS y el cómo los contenedores pueden operar y descubrirse entre ellos. El uso de archivos, como por ejemplo configuraciones para un contenedor en específico y almacenamiento persistente, también es un cambio de paradigma el cual se debe adaptar y realizar de forma correcta para cumplir con el buen funcionamiento.

El no uso de un orquestador de contenedores, a medida que la cantidad de servicios vayan creciendo, puede repercutir a la gobernabilidad de éstos, ya que la trazabilidad, monitoreo y estabilidad se complejiza debido a que el uso de Docker puro para ejecutar contenedores se empieza a ver limitado.

A su vez, la adopción de un orquestador de contenedores puede requerir el contratar otro tipo de perfiles que están en crecimiento, como por ejemplo expertos en Site Reliability Engineering (SRE).

El realizar el cambio de ejecución de contenedores individuales, como por ejemplo en Docker versus la ejecución de éstos de forma orquestada en Kubernetes, representa una mejora en términos de escalabilidad debido a que Kubernetes está diseñado para poder escalar a nivel de nodos y de servicios; además, de su ejecución en *on-premise* o servicios manejados en el Cloud, también en la gobernabilidad debido a que se mantiene un mayor control respecto a los estados de los servicios, la salud de éstos y la granularidad en las configuraciones respecto a recursos asociados, por lo que representa una oportunidad para mejorar la operación de los servicios. Por lo tanto, es altamente deseable realizar esta migración, pero está la dificultad de llevar a cabo exitosamente este proceso.

En este contexto, en este trabajo se postula que el uso de LLM permitiría disminuir horas-persona respecto a tareas como la curva de aprendizaje, migración de configuraciones y componentes, documentación y mayor rapidez en pruebas y despliegues.

1.2 Propuesta solución

Para lograr una migración mas rápida y con menor costo, la propuesta consiste en el uso de LLM (Large Language Model); en este caso en particular, de LLM públicos a través de chatbots como ChatGPT y Bard, como también el uso de LLM como servicio a través de APIs, para asistencia en las configuraciones, validaciones y documentación, sin incorporar personal dedicado exclusivamente a esta actividad.

Actualmente, los modelos LLM públicos están entrenados con gran cantidad de información extraída desde libros, artículos, páginas web, entre otros. Dentro de éstos, se encuentran las páginas web técnicas, como por ejemplo, toda la información que existe respecto a la documentación de software como Kubernetes y Docker, por lo que pueden generar configuraciones y a su vez interpretarlas, como es en el caso que se quiere probar, el de entregar los archivos de configuración de Docker para poder migrar a Kubernetes.

En esta propuesta, debido al uso que se quiere dar con los modelos de LLM, se buscó reducir el tiempo en que una persona deba estar leyendo toda la documentación, para por ejemplo, entender en profundidad cómo van interrelacionadas las distintas configuraciones, sumado al uso de configuraciones estándares, ya que el caso de uso referente a un orquestador de contenedores es algo que ya se encuentra en muchas empresas; el modelo podrá generar también documentación estándar y no será necesaria una mayor personalización de ésta.

En primera instancia se probó la eficacia al convertir configuraciones y pedir documentación utilizando la interfaz gráfica de ChatGPT. Sin embargo, para poder lograr automatización y simplificación a la hora de operar directamente con

configuraciones y otros, se desarrolló una sencilla aplicación que permitió interactuar con la API de OpenAI y así tener mayor flexibilidad a la hora de consultar las distintas configuraciones, documentación y otros.

Para la evaluación de esta investigación se utilizaron pruebas de funcionamiento y seguridad obtenidas a través de buenas prácticas utilizadas en Kubernetes oficial, así como a través de proveedores de nube y también estándares como el NIST.

Como alternativas existentes corresponde a la contratación de empresas externas expertas en el dominio; en este caso, de contenerización, además de contratación de personal interno. Sin embargo, este trabajo está buscando un método con menor costo y mayor rapidez para el avance de esta migración.

1.3 Objetivos

Los objetivos de este trabajo son los siguientes:

- Realizar una migración de microservicios desplegados en contenedores individuales hacia microservicios desplegados en un orquestador de contenedores como Kubernetes apoyado por un LLM público.
- Evaluar la migración realizada con pruebas de funcionamiento y seguridad.
- Creación de una aplicación para facilitar el uso de LLM como servicio a través de API.

1.4 Impacto

El uso de modelos LLM se encuentra actualmente en un estado incipiente y el evaluar esta tecnología en casos reales que puedan aportar a la empresa, a través de menores costos y un mayor *time-to-market* permite que la compañía pueda estar en una posición de ventaja respecto a las demás organizaciones de su área que no estén considerando el uso de estas tecnologías de Inteligencia Artificial.

2. Marco teórico y estado del arte

La realización de este proyecto involucra distintos conceptos que son utilizados para la revisión del problema y la hipótesis planteada, principalmente LLM, contenedores, Docker y Kubernetes.

2.1 Modelos de lenguaje de gran escala (LLM)

Las herramientas como ChatGPT y Bard estrictamente son chatbots de inteligencia artificial y están basados en los llamados modelos de lenguaje de gran escala (del inglés Large Language Models, LLM). Estos modelos son llamados de esa forma debido a que corresponden a modelos de lenguaje pre-entrenados (PLM) pero a gran escala, conteniendo miles de millones de parámetros [2], por lo que se les clasifica dentro de lenguajes de gran escala.

Los modelos pre-entrenados (PLM) utilizan el modelo de Transformer, el cual, para entender su concepción, hay que remontarse al paper “Attention Is All You Need” de A Vaswani et al. (2017), que presenta el modelo de Transformer original, donde propone una arquitectura de red neuronal que utiliza Attention para “aprender” las relaciones entre diferentes partes de una secuencia de datos [3], lo que permitió para temas de traducción entrenar significativamente más rápido el modelo en comparación con el estado del arte de esos años, demostrando que el entrenamientos de modelos con esta técnica podría mostrar resultados en tiempo récord, planeando así también la expansión de estos modelos para otro tipo de tareas.

Luego de la presentación del modelo de Transformer original, se originaron varios papers durante los años siguientes, donde se resalta la importancia del paper “Improving Language Understanding by Generative Pre-Training” de Radford et al. (2018), publicado por OpenAI, que propone un método para mejorar la comprensión del lenguaje natural mediante el pre-entrenamiento generativo (Generative Pre-Training, GPT), el cual corresponde a un enfoque para entrenar modelos de lenguaje

donde en primer lugar se debe entrenar un modelo para generar texto, posteriormente se entrenan los modelos para otro tipo de tareas, como la comprensión del lenguaje [4].

Actualmente el modelo base de ChatGPT utiliza GPT-4 [5], que corresponde a la última versión de GPT, mientras que Bard utiliza LaMDA [6], también basados en el modelo de Transformer.

2.2 Contenedores

El concepto de contenedor es un término adoptado hace muchos años atrás, asociados al comando `chroot` de los sistemas Unix a fines de la década de los 70 [7], el cual fue evolucionando con el tiempo hasta llegar a la tecnología LxC -Linux Containers- disponible en el kernel Linux desde el año 2008. A diferencia de cómo funciona un *hypervisor*, donde se pueden ejecutar distintas máquinas virtuales, cada uno con su sistema operativo sobre un solo host, los contenedores comparten el mismo kernel del sistema operativo y aísla los procesos, binarios y bibliotecas de la aplicación que se encuentra en el contenedor a través de políticas y mecanismos de control de acceso estándares del sistema operativo.

Para el uso de una máquina virtual, se requiere emular hardware e iniciar distintos procesos y servicios asociados a ellos; en cambio, los contenedores se ejecutan en el sistema operativo de forma nativa, sin emulación de por medio, logrando ser mucho más livianos y portables que la ejecución de las máquinas virtuales, pudiendo ejecutarse cientos de contenedores en un mismo host en comparación con un número limitado de máquinas virtuales. Sin embargo, requiere que exista compatibilidad entre los contenedores y el sistema operativo host.

2.3 Docker

Docker es un proyecto open-source que permite una mayor facilidad al momento de desplegar aplicaciones en contenedores. Docker extiende la tecnología LxC a través de una API a nivel de kernel y aplicación, utiliza *namespaces* diferentes para aislar completamente la aplicación dentro del contenedor [7].

El ecosistema de Docker se compone, en primer lugar, de un *daemon* llamado `dockerd`, que es el encargado de manejar los distintos contenedores que son desplegados en el sistema operativo. Para la ejecución de un contenedor, es necesario una imagen de Docker, la cual cuenta con distintas capas de sistema, montadas en solo lectura, donde la última corresponde a la capa montada como escritura, la cual es el contenedor como tal y puede contener estados, además de ser ejecutable [8]. Estas imágenes tienen una sintaxis determinada donde se especifica la construcción, puertos a utilizar, forma de ejecución, entre otros. Lo anterior se encuentra en el archivo `Dockerfile`.

Durante muchos años el uso de Docker como motor de ejecución fue de total dominio; sin embargo, empezó a disminuir a partir del año 2021, cuando Kubernetes decidió adoptar la medida de dejar de utilizar a Docker como *runtime*, específicamente *dockershim*, desde la versión 1.24, reemplazándolo por `containerd` [9].

2.4 Kubernetes

Kubernetes es un orquestador de contenedores, el cual permite la automatización, escalado y administración de aplicaciones en contenedores [10]. En junio de 2014, Google, en el Google Developer Forum anunció Kubernetes, describiéndolo como “Kubernetes es el desacoplamiento de los contenedores de aplicaciones de los sistemas en los que se ejecutan. Google Cloud Platform proporciona un conjunto homogéneo de recursos sin procesar y, a su vez, Kubernetes programa contenedores para utilizar recursos. Este desacoplamiento simplifica el desarrollo de aplicaciones, ya que los usuarios solo solicitan recursos abstractos como núcleos y memoria” [7].

El lanzamiento de Kubernetes generó un revuelo y diferentes empresas —como por ejemplo, Microsoft, VMWare, Amazon y Red Hat—, empezaron a adoptar formas de entregar y compatibilizar con Kubernetes, ofreciendo incluso esta plataforma



como PaaS —tal como es el caso de Azure Kubernetes Service (AKS) y Amazon Elastic Kubernetes Service (EKS)—, los cuales ofrecen una forma de orquestar y manejar contenedores en la nube.

2.5 Chatbot y Prompt Engineering

Los Chatbots son agentes conversacionales automatizados capaces de interactuar con usuarios de alguna plataforma y han demostrado ser útiles en contextos como automatización de tareas, mejorar experiencia de usuario, como por ejemplo, en el servicio al cliente [11]. ChatGPT y Bard son las interfaces de sus modelos LLM en forma de Chatbot, por lo que los usuarios interactúan con estos modelos como si se tratase de una conversación vía Chat, el cual utiliza contextos; por ejemplo, relacionado con la totalidad de la conversación.

Durante la conversación con los Chatbots utilizados por ChatGPT y Bard, se pueden ir consultando y pedir traducciones de textos, todo como un lenguaje natural; sin embargo, también se puede estructurar este texto de entrada para optimizar la eficacia de los LLM y poder convertir esta conversación en estilos como uso de roles, árbol y cadenas de pensamientos, donde esta técnica de estructuración es conocida como *prompt engineering* [12].

2.6 Estado de la Práctica

Durante el año 2023, han aumentado los *papers* relacionados al uso de los LLM, especialmente relacionado al chatbot ChatGPT, en distintas situaciones, ya sea a nivel de educación y tecnología.

En la búsqueda de bibliografía, no se han encontrado investigaciones relacionadas a contenedores y ChatGPT; sin embargo, existen distintos papers relacionados a su desempeño en situaciones relacionadas a la programación, como es el caso del paper “An Analysis of the Automatic Bug Fixing Performance of ChatGPT” [13], en el cual se evalúa ChatGPT en un *benchmark* de corrección de fallos, logrando encontrar que su rendimiento es competitivo en relación a enfoques de aprendizaje profundo, incluso superando su nivel de éxito a la hora de corregir estos fallos.

Adicionalmente, en relación al uso de ChatGPT para otra área en informática, en este caso el de seguridad, en el paper “From ChatGPT to ThreatGPT: Impact of Generative AI in Cybersecurity and Privacy” [14], investiga cómo ChatGPT puede ser usado para crear ataques como *jailbreaks*, ingeniería social, además de automatizar la creación de *malware*, ataques de *phishing*, entre otros. También es utilizado para apoyar las medidas de seguridad; como por ejemplo, automatización, reportería, generación de código seguro e inteligencia, relacionado a la ciberdefensa.

No se encontraron papers relacionados al uso los LLM para migración relacionados a Docker y/o Kubernetes, por lo que esta investigación busca aportar a esta área que es muy utilizada actualmente y, especialmente, entregar el conocimiento de su implementación en una empresa grande que no es específicamente de la industria tecnológica.

3. Contexto y propuesta de investigación

3.1 Contexto de la investigación

Esta investigación se encuentra en el contexto de verificar el estado actual de los LLM respecto a la tarea en particular de la migración de microservicios desplegados en contenedores individuales, como Docker, hacia el despliegue de éstos en un orquestador de contenedores, como Kubernetes. Adicionalmente, se busca ver temas como seguridad, documentación, factibilidad, entre otros. Para lo anterior, se utilizará un proyecto de microservicios creado por el proveedor de Cloud, Google Cloud, llamado Online Boutique [15], que consiste en una demostración del uso de Google Cloud con microservicios en distintos lenguajes de programación.

3.2 Propuesta de investigación

Debido a que se requiere realizar la migración para poder aprovechar todas las ventajas de un orquestador de contenedores, y ya visto el rendimiento obtenido en distintos LLM en el momento de entregar soluciones, es que se quiere utilizar esta alternativa, sin incorporar nuevos roles en la Empresa, como una oportunidad de migración a bajo costo y a la vanguardia de la tecnología. La propuesta de investigación se basa en el uso de estos modelos ya entrenados para documentar y configurar lo obtenido al momento de migrar desde contenedores individuales hacia contenedores orquestados por Kubernetes, utilizando la interfaz de chatbot de ChatGPT, así como ya en un trabajo más avanzado con el uso de las API de estos servicios. Luego, debe ser verificado el rendimiento y eficacia del uso de estos modelos, para verificar las condiciones para una puesta en producción al mundo real.

Esta investigación quiere aportar en el uso de estas nuevas tecnologías como LLM para acelerar y potenciar el uso, crecimiento y adopción de contenedores, además de su preparación para la puesta en producción en nubes públicas y privadas, sin tener que contratar a equipos dedicados y/o externos, logrando potenciar a los roles actuales y apoyándose en la documentación y futuro transferencia de conocimiento generado por las LLM. Esto podría servir a empresas de distinto tamaño y sector para poder avanzar con las pruebas de otras tecnologías, como en este caso, el de un orquestador de contenedores.

3.3 Hipótesis

El uso de LLM permitirá una migración de microservicios desplegados en contenedores individuales hacia un despliegue en un orquestador de contenedores como Kubernetes, generando las configuraciones, documentaciones y pruebas necesarias para su correcto funcionamiento, sin tener que agregar nuevos roles a la compañía, además de un menor tiempo de implementación.

3.4 Variable dependiente

Método de migración de contenedores individuales de Docker a contenedores orquestados por Kubernetes. Básicamente se controla con y sin uso de LLM públicos a través de chatbots y APIs.

3.5 Variable independiente

Eficacia en el uso de LLM públicos a través de chatbots y APIs, en comparación a herramientas tradicionales no guiadas por inteligencia artificial.

3.6 Estrategia de validación

Para validar la hipótesis se requerirá:

- Implementar la solución consistente en el uso de LLM públicos a través de chatbots y APIs para la migración de contenedores individuales de Docker a contenedores orquestados de Kubernetes.
- Implementar la solución con herramientas tradicionales no guiadas por inteligencia artificial.
- Ejecutar pruebas de funcionamiento y seguridad para verificación de la eficacia de la migración.

4. Estrategia de solución e implementación

4.1 Estrategia de solución

El problema central definido en esta tesina corresponde a la complejidad de la adopción de un orquestador de contenedores, en este caso Kubernetes, para el despliegue de servicios web, los cuales actualmente están siendo desplegados directamente a través de Docker instalado como servicio, ejecutando los contenedores de forma individual, por lo que la estrategia de solución se basa en el uso de LLM públicos para disminuir esta complejidad, hacer más eficiente el proceso de migración y lograr ejecutar las aplicaciones requeridas a través de Kubernetes.

En primer lugar, se deben definir los contenedores a migrar, los criterios de selección de estos contenedores y sus configuraciones básicas a traspasar. Posteriormente, se requiere la implementación de Kubernetes, junto a configuraciones internas. Luego viene la sección de definir las pruebas a realizar, los criterios de aceptación de éstas, además de su ejecución ante el despliegue realizado.

Una vez aceptadas las primeras pruebas del uso de LLM públicos a través de Chatbot, se desarrollará una prueba respecto a una aplicación para conectarse directamente a la API que entregará mayor flexibilidad; por ejemplo, para mantener contextos, subida de archivos, entre otros.

Finalmente, se realizará un análisis a las pruebas realizadas, se revisarán temas de seguridad y riesgos, tanto para la migración realizada por las respuestas de los chatbots como de la aplicación desarrollada para este fin.

4.2 Definición de criterios de selección de las aplicaciones en contenedores a migrar

Las aplicaciones a utilizar en este proyecto deben considerar los siguientes criterios:

- Criterio 1: Ser aplicaciones ya creadas en contenedores. No se tomará el caso del traspaso de una aplicación/servicio que se encuentre en máquinas virtuales de forma directa, sino que ésta ya se encuentre contenerizada.
- Criterio 2: Arquitectura de microservicios. La aplicación debe estar construida según la arquitectura de microservicios, el cual cuenta con distintos servicios que se ejecutan de manera independiente y se conectan entre sí, a diferencia de una aplicación monolítica.
- Criterio 3: Aplicación de código abierto con licenciamiento libre. En esta ocasión, para poder manipular la aplicación y sus componentes, se requiere que no existan conflictos a la hora de poder modificar parte de la aplicación.
- Criterio 4: Aplicación de conocimiento público y vigente. Para que los resultados puedan ser reproducidos por cualquier persona, además de asegurar continuidad en relación a que la aplicación no será dejada de usar (*deprecated*) en el corto tiempo, se debe escoger una aplicación que esté en continuo desarrollo y tenga soporte vigente.
- Criterio 5: Verificación de funcionamiento de aplicaciones en Kubernetes. Para continuar con el enfoque de esta investigación, respecto a la migración utilizando LLM, se requiere que la aplicación ya tenga configuraciones de Kubernetes realizadas para su comparación.

Con los criterios establecidos anteriormente, se espera poder contribuir con un caso cercano a una implementación real de un servicio web en un ambiente empresarial, el cual contiene una arquitectura moderna de aplicaciones, además de no existir restricciones en su modificación y, por último, que las configuraciones y documentación realizada por los LLM permitan ser verificados.

4.3 Definición de criterios de pruebas a realizar

Para asegurar que la migración realizada desde los contenedores individuales hacia el orquestador de contenedores sea válida, se realizaron distintas pruebas para verificar su eficacia:

- Prueba 1: Los microservicios deben ser capaces de ser ejecutados y de proveer las funcionalidades como si fueran ejecutadas como contenedores individuales; es decir, proveer el servicio web para el que fue diseñado.
- Prueba 2: Los microservicios deben ser capaces de conectarse entre sí para proveer las funcionalidades para lo que fueron programados; es decir, debe existir conectividad entre ellos y poder funcionar correctamente.
- Prueba 3: La página web que contiene en su totalidad todos los microservicios debe funcionar en forma completa; es decir, capaz de proveer las mismas funcionalidades que como si fuese ejecutado en la manera antes de la migración.
- Prueba 4: Los microservicios deben ser capaces de escalar de forma automática cuando éstos sean puestos en prueba de estrés, permitiendo así uno de los beneficios al ser orquestados a través de Kubernetes.
- Prueba 5: La configuración y documentación de los microservicios deben ser acorde a las validaciones de las configuraciones ya realizadas por el proveedor, además de tener documentación referente a la migración realizada a Kubernetes.

Con las pruebas descritas anteriormente, se puede asegurar el correcto funcionamiento de la aplicación en su totalidad. Además, poder ser replicados, validados respecto a sus configuraciones y tener la documentación para entender cómo se encuentra funcionando la aplicación y sus componentes.

4.4 Definición de parámetros de seguridad respecto a la migración

Las configuraciones asociadas a los microservicios en el orquestador de contenedores, así como las referente a la configuración del cluster de Kubernetes en sí, se evaluaron para verificar la robustez de los parámetros indicados por la LLM, así como también el poder aconsejar acerca de las buenas prácticas en la industria. Para lo anterior, se busca seguir buenas prácticas en la industria, documentadas según los frameworks NSA-CISA [16], MITRE ATT&CK [17] y CIS Benchmark [18], los cuales son referentes respecto al *hardening* de Kubernetes y microservicios.

- Parámetro 1: Plano de Control. Éstos son los componentes que se encargan de mantener los contenedores funcionando, recibiendo llamadas, verificando el estado de los componentes, entre otros.
- Parámetro 2: Redes. Corresponde a la infraestructura que permite la conectividad de manera interna y externa entre contenedores, cluster y usuarios finales.
- Parámetro 3: Workloads. Corresponde a el o los contenedores que se encuentran siendo ejecutados en *pods* de Kubernetes.

Con los parámetros establecidos anteriormente, se puede asegurar en parte que el cluster y los contenedores se encuentran ejecutados en un ambiente con mejores prácticas, con configuraciones válidas en términos de infraestructura y con resguardos básicos a nivel de seguridad.

4.5 Implementación de infraestructura necesaria

Dentro de los aspectos referente a la infraestructura donde se ejecutaron el cluster de Kubernetes, en esta solución se utilizaron dos versiones. La primera corresponde a la utilización de Minikube [19] que permite una instalación local de manera rápida y sin complicaciones, utilizado preferentemente para la realización de pruebas. La segunda corresponde al despliegue de un cluster de Kubernetes en el proveedor de Google Cloud, llamado Google Kubernetes Engine [20] el cual es uno de los más

utilizados por el mercado y cuenta con varias características para soportar distintas cargas, además de una completa integración en el ecosistema de Google Cloud.

Referente a la implementación de cómo se recibirá el tráfico desde el exterior del cluster, se utilizarán los siguientes componentes de Kubernetes:

- LoadBalancer: Es un componente en Kubernetes que permite proveer acceso de red externa hacia el interior del cluster.
- Ingress: Es un componente en Kubernetes que permite rutear tráfico desde fuera del cluster hacia los servicios dentro del clúster.

4.6 Implementación de aplicación para apoyo en migración

Debido a que OpenAI, la empresa detrás de ChatGPT, permite el uso de una API para acceder al LLM, no solo a través de un chatbot como es ChatGPT, se implementó una aplicación que permitió una interacción adaptada para este escenario de migración desde contenedores locales hacia el orquestador de contenedores, la cual contiene los siguientes diferenciadores respecto a escribir todas las instrucciones a través de ChatGPT:

- Contexto permanente: Todas las instrucciones enviadas al modelo de OpenAI van con un contexto referente al rol que el sistema debe mantener, en este caso, el rol de asistente de migración.
- Facilidad de uso: El programa está adaptado para seleccionar los archivos *Dockerfile* a migrar, a diferencia de estar manualmente copiando estos datos e ingresarlos en ChatGPT, son leídos por el programa y enviados directamente a la *API* de OpenAI.
- Interactividad: Dependiendo de la acción dentro de la migración que se está realizando, el programa viene con *prompt* predefinidos para ir guiando al usuario que se encuentra utilizando la aplicación.

Finalmente, con el uso de esta aplicación, el usuario que desea realizar la migración no necesita estar capacitado en *prompt engineering* porque el sistema viene con todo lo relacionado de forma predefinida.

5. Resultados

La migración realizada por las respuestas de los chatbots como de la aplicación desarrollada para este fin.

5.1 Selección de las aplicaciones en contenedores a migrar

De acuerdo a los criterios establecidos, se escoge el proyecto de microservicios creado por el proveedor de Cloud, Google Cloud, llamado Online Boutique [15], que consiste en una demostración del uso de Google Cloud con microservicios en distintos lenguajes de programación. Este cumple con los criterios asignados anteriormente:

- Criterio 1: El proyecto ya cuenta con los microservicios de forma contenerizada, de forma que cada uno de éstos tiene un *Dockerfile* asociado.
- Criterio 2: El proyecto tiene la arquitectura de microservicios, por lo que cada servicio que es expuesto es independiente del otro; como por ejemplo, el servicio de pagos con el servicio de carrito de compras están incluso implementados en distintos lenguajes de programación.
- Criterio 3: El licenciamiento del proyecto es Apache 2.0, el cual es una licencia de software libre y permite al usuario usar el software con cualquier propósito, además de no tener que entregar retribución alguna.

- Criterio 4: El proyecto se encuentra activamente desarrollado desde 2018, cuenta con mas de 15 mil estrellas en GitHub y es utilizado para mostrar distintas tecnologías en Google Cloud.
- Criterio 5: Actualmente, este proyecto cuenta con los archivos necesarios para ser desplegados en Kubernetes, por lo que se podrá hacer una comparación entre los archivos resultantes por el LLM y los archivos con los que fueron diseñados para ser ejecutados.

La versión utilizada de la aplicación seleccionada corresponde a la 0.8.0, lanzada el 15 de junio de 2023. La estructura de la aplicación se muestra en la Figura 1.

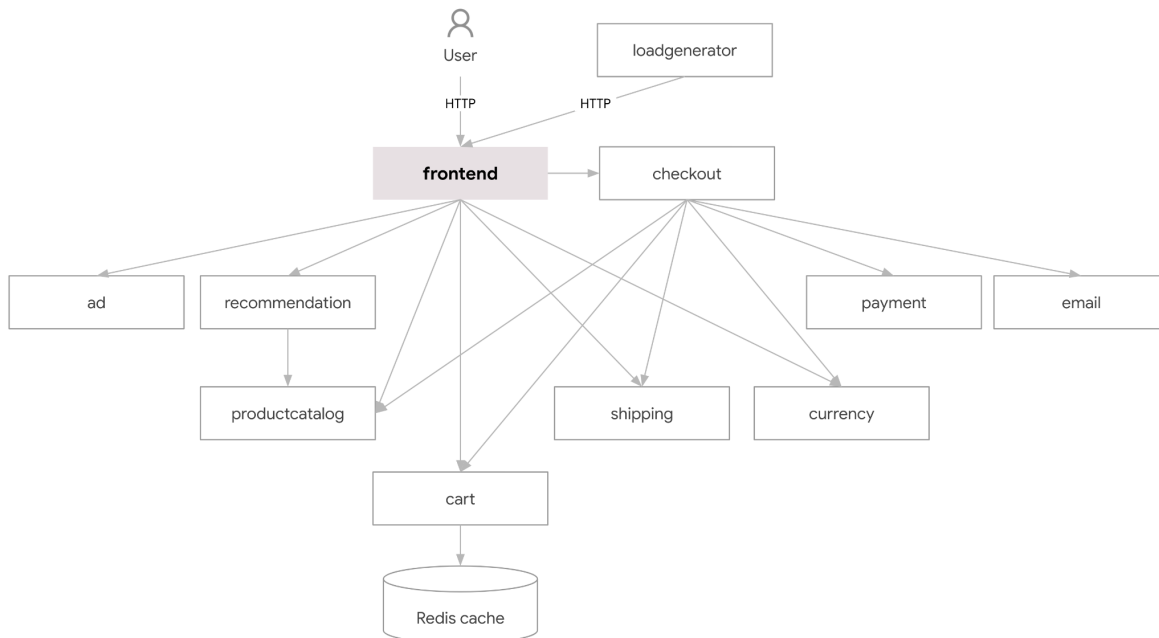


Figura 1. Estructura de aplicación “Online Boutique”.

Además, cuenta con los servicios descritos en la Tabla 1.

Tabla 1. Descripción microservicios de “Online Boutique”.

Servicio	Lenguaje	Descripción
frontend	Go	Expone un servidor HTTP para servir el sitio web. No requiere registro/inicio de sesión y genera automáticamente IDs de sesión para todos los usuarios.
carservice	C#	Almacena los artículos en el carrito de compras del usuario en Redis y los recupera.
productcatalogservice	Go	Proporciona la lista de productos desde un archivo JSON y la capacidad de buscar productos y obtener productos individuales.
currencyservice	Nodejs	Convierte una cantidad de dinero a otra moneda. Utiliza valores reales obtenidos del Banco Central Europeo. Es el servicio con mayor cantidad de peticiones por segundo (QPS).

paymentservice	Nodejs	Carga la información de la tarjeta de crédito proporcionada (simulado) con el monto dado y devuelve un ID de transacción.
shippingservice	Go	Proporciona estimaciones de costo de envío basadas en el carrito de compras. Envía los artículos a la dirección proporcionada (simulado).
emailservice	Python	Envía a los usuarios un correo electrónico de confirmación de pedido (simulado).
checkoutservice	Go	Recupera el carrito del usuario, prepara el pedido y orquesta el pago, el envío y la notificación por correo electrónico.
recommendationservice	Python	Recomienda otros productos basados en lo que se encuentra en el carrito.
adservice	Java	Proporciona anuncios de texto basados en las palabras de contexto dadas.
loadgenerator	Python	Envía continuamente solicitudes imitando flujos de compra de usuarios realistas al <i>frontend</i> .

5.2 Resultados referente a interacción con ChatGPT

Para la interacción con ChatGPT, en primera instancia se dictó un *prompt* específico para comenzar a realizar la tarea respecto a la migración. El *prompt* tiene una estructura específica que se puede ver a continuación:

- “Tu eres un bot de asistencia experto en Docker y Kubernetes. Tu misión es ayudar respecto a la conversión de microservicios separados en Docker hacia microservicios en Kubernetes. En primer lugar debes consultar acerca del microservicio a convertir y su Dockerfile, luego debes entregar los comandos para la creación de una imagen, subida de esta y posteriormente los archivos para su ejecución. En segundo lugar debes entregar el comando para que el usuario pueda realizarlo en una terminal Linux con kubectl instalado. Debes esperar a que te entregue el output del comando ejecutivo para seguir con el análisis. Debes verificar el estado del microservicio implementado e indicar alguna prueba a realizar al usuario. Si ya está todo bien, debes seguir consultando por el siguiente microservicio. Al final, si el usuario indica que ya está todo realizado, entrega un resumen de lo implementado.”

El *prompt* está compuesto en primer lugar asignándole un rol y especificando un objetivo. Luego, se le asigna el cómo va a interactuar, qué cosas debe tener en consideración, cómo debería ir procediendo, lo que se espera del usuario y, finalmente, una acción final. ChatGPT responde al mensaje, indicando que seguirá las instrucciones por lo que se encuentra a la espera de los microservicios junto los *Dockerfile* correspondientes (ver Anexo 1).

A través de la web, se copia el contenido del archivo *Dockerfile* y luego se ingresa en el *prompt*, indicando que corresponde al primer servicio junto a su nombre, volcando el archivo hacia el chatbot (ver Anexo 2).

ChatGPT analiza el *Dockerfile* entregado, y gracias al manejo del contexto, entrega los pasos para la creación y subida de la imagen, junto al despliegue en Kubernetes (ver Anexos 3, 4 y 5).

Para el último paso, ChatGPT, debido a la instrucción dada al comienzo referente a *Debes esperar a que te entregue el output del comando ejecutivo para seguir con el análisis*, entrega la instrucción acerca de proporcionar la salida del comando utilizado para ver la estadística relativos a los *pods* que se encuentran desplegados (ver Anexo 6).



Una vez ejecutado los comandos, se entrega el *output* a ChatGPT para revisar el estado, en el cual éste tiene en consideración el chequeo tanto del *pod* como de los servicios (ver Anexo 7)

Este ejercicio se realiza con todos los servicios de la misma manera; es decir, entregando los *Dockerfile* a ChatGPT, ejecutando los comandos en el terminal y entregando el *output*. Es importante recordar que la ejecución de comandos en el terminal directo desde fuentes externas no es recomendado en un ambiente productivo, sino que se anima a su ejecución en ambientes de prueba, como es el caso en estos ejercicios.

Algunos ajustes que se realizaron en el intertanto de la conversación y que fue agregado en contexto, fueron relacionados con la versión de Kubernetes. En este caso, la prueba se realizó con *minikube*, por lo que en un primer momento ChatGPT entregó comandos que estaban pensados para Kubernetes del proveedor en la nube, como es el caso de la manipulación de los objetos *LoadBalancer*, que permite enviar tráfico externo a los contenedores, de forma balanceada. La entrega de comandos que no funcionan en la versión de Kubernetes que se probó, puede ser un bloqueador para el personal sin conocimientos de Kubernetes; sin embargo, al entregar mayor contexto a ChatGPT, éste es capaz de entregar información mas precisa, por lo que finalmente entregó los comandos correctos.

Otro punto a considerar corresponde a dónde se guardan las imágenes. ChatGPT entregó por defecto el uso de imágenes en un proveedor público, el cual corresponde a *DockerHub*, por lo que se le pidió los comandos para implementar un repositorio de imágenes privado, el cual se puede ejecutar de forma sencilla y sin problemas dentro de Kubernetes. Considerando que el personal pudiese haber hecho uso de este proveedor público de imágenes, hubiese existido riesgo en la seguridad al publicar imágenes pertenecientes a la empresa, con posible información confidencial, hacia la nube.

5.3 Resultados obtenidos de las pruebas

De acuerdo a los criterios de definición de las pruebas respecto a la utilización de LLM, se obtuvieron los siguientes resultados:

5.3.1 Prueba 1

Al momento de entregar el archivo *Dockerfile*, éste contenía el parámetro de *EXPOSE*, el cual sirve para indicar en qué puertos el contenedor debe escuchar, lo que es interpretado para ChatGPT como usarlo en el objeto *Service*, redirigiendo el tráfico del puerto 80 (comúnmente utilizado para el protocolo *HTTP*) hacia el puerto indicado en *EXPOSE*, por lo que en términos de redes, los microservicios lograron ser ejecutados correctamente. Sin embargo, no todos los microservicios lograron ser ejecutados en su primer intento, debido a que algunos requieren de parámetros adicionales, como por ejemplo, las variables de entorno, los cuales sirven para indicar ciertas configuraciones y éstas puedan ser leídas desde el microservicio. Un ejemplo de variable de entorno a considerar, es la dirección de otro microservicio, como por ejemplo, bajo qué dominio y en qué puerto se expuso el servicio de carrito de compras, el de pagos, entre otros.

Siguiendo la hipótesis de esta investigación, se buscó ayuda a ChatGPT para la solución de problemas, por lo que se reportó el estado de *'CrashLoopBackOff'*, el cual es un estado en Kubernetes que reporta que el contenedor está iniciándose constantemente (ver Anexo 8). Por consiguiente, ChatGPT sugiere comandos para verificar el estado del contenedor, por lo que pidió el *output* para poder ser analizado (ver Anexo 9).

ChatGPT logra detectar de forma correcta cuál es el problema, debido a que el microservicio tiene como requisito definir la variable de entorno para poder funcionar, ya que si no es entregada, el servicio no se inicia, quedando en este estado de constante reinicio. Si bien el diagnóstico es correcto, la solución entregada no es óptima debido a que los servicios pueden ser publicados de forma dinámica, por lo que la inclusión de esta variable de entorno se recomienda en el momento de la ejecución del contenedor y no en el momento de la creación de la imagen.



Una vez resuelto los problemas y continuando con el despliegue de servicios, se entrega el output final a ChatGPT, el cual detecta que el contenedor ya no se encuentra en un estado de *'CrashLoopBackOff'*, sino que en *Running* (ver Anexo 10).

Por lo tanto, con algún trabajo mayor en algunos microservicios, ChatGPT logra pasar la prueba respecto a que los microservicios deben ser capaces de ser ejecutados y de proveer las funcionalidades como si fueran ejecutadas como contenedores individuales; es decir, proveer el servicio web para el que fue diseñado.

5.3.2 Prueba 2

En el momento del despliegue, ChatGPT entregó por cada microservicio dos configuraciones, las cuales consisten en los objetos *Deployment* y *Service*, y éstos fueron aplicados sin configuración adicional alguna. Los microservicios obtuvieron dirección IP a través del objeto *Service*, lo que permite exponer los servicios y, a su vez, se asigna un DNS interno que es utilizado para la conexión entre microservicios (ver Anexo 11). Para probar la conectividad entre servicios, se pueden ejecutar comandos dentro del contenedor y así verificar el estado de éstos, como por ejemplo ejecutar *grpcurl* para hacer una consulta al servicio *GRPC* expuesto (ver Anexo 12).

En la conectividad que se probó, se verifica conexión desde el microservicio *cartservice* hacia el microservicio *adservice*. Se hicieron pruebas con mas microservicios y la conexión funcionó correctamente, por lo que no se necesitó de información adicional a lo entregado por ChatGPT.

5.3.3 Prueba 3

La siguiente prueba consiste en tener todo el proyecto funcionando en Kubernetes con las configuraciones entregadas por ChatGPT, por lo que se debe cumplir el objetivo de escoger un producto, revisarlo en detalle junto a la conversión de los precios, luego agregar al carro, para finalmente crear una orden con una tarjeta de crédito asociada.

El inicio de la página actualmente es provisto por el microservicio de *frontendy* en él muestra productos que se encuentran disponibles por esta tienda virtual, además indica la moneda de pago del servicio que por defecto viene en dólares (ver Anexo 13). Para realizar la prueba, se seleccionó el producto correspondiente a *Sunglasses* y se seleccionó la moneda *GBP* para la prueba del ratio de conversión, provisto por el microservicio *currencyservice* (ver Anexo 14).

Una vez agregado al carrito de compras, de forma automática se rellena información simulada de datos personales, de dirección y de información de pagos (ver Anexo 15). Cuando la orden se realizó, se pudo revisar el correcto funcionamiento de algunos microservicios a través de los *logs* de cada uno de éstos, como es en el caso de los microservicios de envío de correo y del servicio de pago (ver Anexos 16 y 17).

Finalmente, se pudo probar todo el ciclo de vida respecto a la compra de un producto, por lo que con las configuraciones adicionales que se describieron referente a la configuración de las variables de ambientes, no se requirió información adicional alguna para el correcto funcionamiento del servicio web completo.

5.3.4 Prueba 4

Para la ejecución de las pruebas de escalabilidad, se utilizó el programa *locust*, el cual permite generar peticiones a sitios web para probar los distintos *endpoints* que se exponen por la aplicación. El programa utilizado recibe los parámetros referente a la cantidad de usuarios a simular y la tasa de aparición de estos usuarios. En esta ejecución se utilizaron los parámetros de 50.000 usuarios con una tasa de crecimiento de 100, junto con la ejecución de 5 instancias independientes de *locust*.



En los primeros momentos de ejecución de *locust*, éste indica el tipo de petición *HTTP* realizada, el *path* correspondiente y estadística respecto a la cantidad de peticiones realizadas, fallidas, y más (ver Anexo 18).

Se registra que en un comienzo la carga de uso de CPU y memoria es baja, debido que no existía petición alguna momentáneamente (ver Anexo 19). También se registran los cinco contenedores de *loadgenerator*, los cuales corresponden a la aplicación de *locust*, que fue ejecutada en el mismo clúster de Kubernetes.

Después de un tiempo de ejecución del generador de carga, se puede apreciar como algunas instancias de *locust* empiezan a reportar fallas al momento de realizar las peticiones correspondientes hacia la aplicación web *Online Boutique*. En este caso, se verifica que un 97% de las peticiones se encuentran con fallas (ver Anexo 20). Debido a la repercusión en el clúster respecto a la gran carga de *locust* hacia los microservicios, estos aumentan considerablemente el uso de CPU y memoria, específicamente para los microservicios de *currency* y *frontend* (ver Anexo 21). Sin embargo, ninguno de éstos escaló para poder recibir de mejor manera la carga que se les está transmitiendo.

Al verificar los registros del microservicio de *frontend*, comienza a reportar errores de comunicación, como por ejemplo, respecto a que, no logra conectarse al microservicio de ads de forma satisfactoria, debido a la intensa carga que se está realizando con *locust* (ver Anexo 22).

Para una experiencia de usuario real, se ingresa al sitio web que provee *frontend* y se revisa la información utilizando las *DevTools* de *Chrome*, donde se registra el acceso al sitio y se espera hasta que el servidor devuelva alguna respuesta. Al realizar esta acción durante la prueba de carga, éste demora mas de dos minutos en responder, tiempo suficiente para que en muchos servicios ya se dé por perdida la comunicación (ver Anexo 23).

Finalmente, una vez detenidas las instancias del generador de carga, se observó que el uso de CPU y memoria quedaron en alza en el microservicio de *frontend*, además de estar sin respuesta a pesar de aumentar su uso desde 6 MB de memoria hasta 9411 MB (ver Anexo 24). Los microservicios no fueron desplegados con límite de CPU y memoria, lo que no corresponde a una buena práctica y además en ciertos escenarios podría incluso comprometer la estabilidad del clúster por completo, además de quedarse gastando mucho mas dinero de lo presupuestado debido a que el cobro es de CPU y memoria por minuto en los proveedores Cloud.

Para completar esta prueba, se le consultó acerca de esta problemática de no escalamiento de los microservicios a ChatGPT para encontrar una solución, el cual entrega la explicación referente a utilizar el objeto *HorizontalPodAutoscaler* (*HPA*) que permite aumentar la cantidad de réplicas de un contenedor dependiendo de ciertos factores, como por ejemplo, el uso de CPU, el cual es entregado junto a un rango de mínimos y máximos de réplicas que estarán permitidos en el clúster (ver Anexo 25).

Sin embargo, existieron algunas incompatibilidades respecto a la versión de *autoscaling* correspondiente a la que se está utilizando en *minikube*, por lo que después de un par de intercambios con ChatGPT, entregó una solución la cual funcionó para crear el objeto de *HorizontalPodAutoscaler*; sin embargo, se registraron algunos errores (ver Anexo 26).

Nuevamente se realiza la consulta referente al error, el cual tiene relación a que al momento de definir la configuración del microservicio, en éste no se especificaron los *requests* y *limits* de CPU y memoria que son utilizados para definir la memoria requerida para la ejecución del contenedor y los límites que debiese tener, además de ser parámetros necesarios para contar con un punto de medición y así activar *HPA* (ver Anexo 27).

Una vez establecida la configuración en los microservicios de *currencyservice* y *frontend*, además de sus respectivas configuraciones de *HPA*, se ejecutó nuevamente el simulador de carga, donde se verificó que solo el servicio de *currencyservice* estaba escalando, mientras que el servicio de *frontend* se encontraba con un error, llamado *OOMKilled* (ver Anexo 28). Este error consiste en que el contenedor fue eliminado debido a traspasar los límites respecto a la memoria RAM.

Debido a que el microservicio de *frontend* se encontraba con problemas al superar los límites en memoria RAM, y considerando que el *HPA* activado solo medía la utilización de CPU, se intentó consultar esto con ChatGPT el cual debido a su limitación respecto al tiempo en que fue entrenado, no estaba al tanto de la nueva versión de la API que se encuentra en Kubernetes v1.28, el cual utiliza la versión 2 de *autoscaling* (ver Anexo 29). Sin embargo, al momento en que fue entrenado ChatGPT, éste solo llegaba a la versión *v2beta2*, la cual quedó obsoleta en la versión de Kubernetes que se estaba utilizando, por lo que la configuración entregada no podía ser utilizada sin realizar modificaciones.

Por lo tanto, para lograr lo requerido en esta prueba, se buscó una versión de la configuración compatible con la versión 2, el cual logró ejecutar con éxito el escalamiento según la cantidad de memoria RAM observada y no solamente de CPU como ocurría en la versión 1 de *autoscaling*. Con la configuración referente al *HPA* ya en ejecución, se pudo verificar una cantidad de 9 réplicas respecto al *currencyservice* y de 3 réplicas respecto al *frontend*; mientras que al verificar el estado de los contenedores en ejecución, se pudo comprobar la cantidad de réplicas existentes, además del uso distribuido en CPU y memoria en las diferentes réplicas (ver Anexos 30 y 31).

Resumiendo los resultados de este ejercicio, se avanzó considerablemente con la ayuda de ChatGPT para encontrar una solución respecto al escalamiento; sin embargo, debido a no encontrarse actualizada, no logró resolver completamente el cometido de soportar una carga que permita el autoescalamiento de los contenedores. Además, cabe destacar que existen distintas formas de optimizar este proceso de encontrar el escalamiento ideal dependiendo de la carga futura, el aplicativo en cuestión y su uso. Sin embargo, por motivos de evaluar el rendimiento del LLM es que se simplificó a una métrica genérica respecto al uso de CPU y memoria.

5.3.5 Prueba 5

Una vez realizadas las pruebas 1 a 4, se verificó el archivo creado por el proveedor respecto al archivo que entregó finalmente ChatGPT a lo largo de su implementación. Para esto, se utilizó la herramienta *diff* entre los archivos de configuración mencionados respecto al microservicio de *frontend*.

Se verificó que el archivo del proveedor contiene mas configuraciones (ver Anexo 32) entre las cuales se pueden destacar:

- *securityContext*: Utilizado para ejecutar el contenedor con menos privilegios y evitar ataques al cluster completo.
- *readinessProbe*: Utilizado para indicar cuándo el contenedor se encuentra disponible para recibir tráfico.
- *livenessProbe*: Utilizado para indicar cuando el contenedor se encuentra en buen estado.

Al consultar acerca de estos parámetros a ChatGPT, éste entrega información acerca de éstos, además de indicar cómo se puede utilizar en el archivo de configuración.

Finalmente, se puede verificar que el trabajo realizado por ChatGPT al momento de generar estas configuraciones cuenta con lo necesario para la ejecución del contenedor; sin embargo, otras configuraciones referente a mejores prácticas como entregar el privilegio mínimo para funcionar y la utilización de estas sondas para verificar estado de los contenedores no son entregados a menos que se consulten específicamente acerca de éstos, lo que podría ser desconocido para quienes estén realizando esta migración sin conocimiento detallado de Kubernetes.

5.3.6 Documentación

Uno de los aspectos que se deben considerar, especialmente en entornos de trabajo en equipo, es la necesaria documentación de lo que se encuentra en ejecución; es decir, mantener un registro con la explicación de las configuraciones, indicaciones acerca de configuraciones específicas, herramientas utilizadas, además de tener pasos que permitan reconstruir desde cero en caso de problemas para poder seguir funcionando.

En este caso, se pidió a ChatGPT la creación de un documento explicando los distintos archivos de configuración de Kubernetes, además de su justificación, lo que permitió una manera rápida e intuitiva de generar la documentación pertinente para su uso, el cual se puede contemplar una muestra en el Anexo 33.

5.4 Resultados obtenidos respecto a seguridad

Para la realización de pruebas referente a seguridad en el clúster, se utilizó como herramienta principal *kubescape* [21] que permite analizar un cluster de Kubernetes y todos sus componentes en relación a buenas prácticas, seguridad, cumplimiento y detección de malas configuraciones. La información de cada control se verificó en la página de *kubescape*, además se le entregó información a ChatGPT acerca de qué es lo que realiza esta herramienta. El programa entrega el título y un enlace con la descripción de cada ítem, donde éste se explica en qué *framework* de seguridad está, el nivel de severidad, recursos asociados, entre otros (ver Anexo 34).

5.4.1 Prueba 1

Referente a las pruebas del Plano de control, se encontraron algunas configuraciones inseguras, las cuales fueron entregadas a ChatGPT para buscar una solución y mitigar estas posibles vulnerabilidades:

- Acceso anónimo se encuentra habilitado.
- Los registros de auditorías se encuentran deshabilitados.
- La encriptación de *etcd* se encuentra deshabilitada.

Respecto a la primera configuración insegura, se verificó en la documentación que se trata de recursos *ClusterRoleBinding* o *RoleBinding* que entreguen acceso anónimo, además de verificar si el clúster tiene habilitado el *flag --anonymous-auth*. Si bien se consultó con ChatGPT, ayudando también a entender de qué se trataba este control, los comandos entregados para ser ejecutados y mitigar estas configuraciones, no resultaron en forma exitosa, por lo que se mantuvo activa esta advertencia.

Referente a los registros de auditorías y la encriptación de *etcd*, se probó parte de lo entregado para solucionarlo pero al igual que en el punto anterior, no se logró mitigar de forma exitosa.

5.4.2 Prueba 2

Los resultados encontrados respecto a parámetros relacionados con la red dentro del clúster (ver Anexo 35). En éste se encontró que los microservicios desplegados no contaban con políticas de red, lo que significa que el tráfico que entra y sale de los contenedores no se encuentran controlados, lo que puede convertirse en una vulnerabilidad de seguridad.

La mitigación de este control corresponde a crear políticas de red para cada microservicio, por lo que luego de entregar la información, además de indicar que se requiere conexión desde *frontend* hacia los demás microservicios, entregó una nueva configuración con el objeto *NetworkPolicy* (ver Anexo 36). Cabe destacar que se debe indicar la orden exacta, como en este caso que se tuvo que reforzar que la conexión va desde *frontend* hacia los demás microservicios.

Al ejecutar el comando y así crear este nuevo objeto de *NetworkPolicy* se probó la aplicación, la cual seguía funcionando y luego al realizar el análisis, ya no mostró el servicio de *frontend* dentro de los servicios con falta a las políticas de red, por lo que se redujo el número de recursos con fallas (ver Anexo 37).

En términos de esta prueba realizada, si bien las configuraciones entregadas por ChatGPT no entregaron políticas de red al momento de implementar, si logró entregar estas configuraciones una vez que se consultó de forma adicional.

5.4.3 Prueba 3

A continuación se revisarán los resultados respecto a las pruebas de seguridad que corresponden a los *Workloads*, es decir, lo relacionado a cómo los microservicios se encuentran configurados y su ejecución. Para ello, nuevamente se utiliza *kubescape*, en este caso, muestra que existen contenedores que se encuentran siendo ejecutados con permisos de *root*, lo que en caso de alguna vulnerabilidad en el servicio que estén ofreciendo, podría tener un impacto más grande debido a que el atacante obtendrá todos los permisos en el contenedor vulnerado (ver Anexo 38).

Revisando con mayor detalle, aparece mencionado el microservicio de *frontend* como afectado, el cual como se indicó en la prueba 5.3.5, éste se encuentra siendo ejecutado como usuario *root* y no cuenta con las configuraciones adecuadas (ver Anexo 39).

En este caso, la aplicación entrega información de cómo remediar este problema en la sección “*Assisted Remediation*”. Sin embargo, se reportó a ChatGPT y entregó una nueva configuración incorporando los parámetros necesarios para ejecutar el contenedor como otro usuario distinto a *root*. Se ejecutó la nueva configuración, luego se verificó el funcionamiento de la página web y se analizó nuevamente con *kubescape*, donde reportó un recurso menos dentro de los afectados, por lo que en este caso la mitigación funcionó de forma efectiva (ver Anexo 40).

5.5 Resultados con migración sin LLM.

Como base de comparación principal respecto a una migración sin apoyo de *LLM*, se buscó una alternativa que permita convertir los contenedores individuales hacia poder ejecutarlos en *Kubernetes*, dentro de las principales propuestas siendo a través de empresas de consultoría. Sin embargo, se encontró una alternativa que necesita de un paso intermedio el cual consiste en ejecutar *Docker Compose* [22] que es una herramienta para definir y ejecutar aplicaciones con multi contenedores, como es el caso de la aplicación escogida en este trabajo, creada por el equipo de *Docker*. Luego, una vez obtenida la configuración que permite la ejecución de la aplicación con *Docker Compose*, se procedió a utilizar una herramienta que es parte del grupo de *Kubernetes*, llamada *Kompose* [23] que permite la migración desde archivos de configuración de *Docker Compose* hacia archivos de configuración de *Kubernetes*.

5.5.1 Migración de contenedores individuales a Docker Compose

Docker Compose permite ejecutar los distintos contenedores como un solo *stack* donde a través de una configuración permite facilitar el manejo a nivel global de la aplicación. Para comenzar a utilizar esta herramienta, se debe instalar un *plugin* en *Docker* y luego crear la configuración para ser ejecutada. Con unos pocos comandos es posible construir las imágenes de los contenedores a utilizar y también iniciar los servicios para su ejecución.

Al seguir la documentación, éste indica que se deben tomar ciertos parámetros que se pueden encontrar en el *Dockerfile* como el puerto de exposición, además de la ubicación del *Dockerfile* correspondiente para la construcción de la imagen. El proceso es sencillo, sin embargo se deben realizar ajustes, especialmente referente a las variables de ambientes, los cuales deben ser especificados para poder ejecutar las aplicaciones que lo requieran.

Después de una hora se obtuvo la configuración base de *Docker Compose*, el cual cuenta con la siguiente estructura: Especificación del servicio, imagen o directorio con el código de fuente que contiene el archivo *Dockerfile*, puertos a exponer y variables de ambientes (ver Anexo 41).

Finalmente, se realizaron unas pruebas y el sitio funcionó correctamente a nivel básico (ver Anexo 42), es decir, no se realizaron pruebas de seguridad ni de rendimiento debido a que correspondía a un paso intermedio.

5.5.2 Migración desde Docker Compose hacia Kubernetes

La herramienta que se utilizó para la migración de Docker Compose hacia Kubernetes corresponde a Kompose, la cual en su documentación muestra que es una forma fácil y sencilla de migrar, tan solo ejecutando la herramienta incorporando como entrada el archivo de configuración de Docker Compose y obteniendo como salida los archivos de configuración para ser desplegados en Kubernetes, generando objetos como *Deployment*.

Al ejecutar el comando, el programa crea los archivos de configuración que corresponden a los distintos microservicios definidos con los objetos de *Deployment* y *Service* (ver Anexo 43).

La herramienta crea lo necesario para ejecutar el microservicio, es decir, los puertos en los que se expondrá el tráfico, además de las variables de entorno. Para que Kubernetes pueda ejecutar los contenedores, requiere que las imágenes usadas por éstos, estén en un *registry*, por lo que se debe especificar un comando adicional en la herramienta para poder construir la imagen y además subirla al *registry* deseado.

Al probar la aplicación a través del despliegue realizado de los archivos de Kubernetes creados por la herramienta Kompose, funcionó correctamente.

Finalmente, cabe destacar que al igual que en la primera versión desplegada por los archivos entregados por ChatGPT, se tuvieron que realizar unos comandos adicionales respecto a las imágenes y en ambos casos la seguridad base de los microservicios y del cluster de Kubernetes se mantuvieron en el mismo estado.

5.6 Resultados con aplicación dedicada

Actualmente el uso de ChatGPT es sólo una de las interfaces que se utiliza para contactar al LLM, este a modo de chatbot, debido a que también se puede contactar directamente a través de API. En este caso, se utilizó la API de OpenAI para crear una capa que ofrece una mayor personalización, a diferencia de la utilización de ChatGPT para acceder al LLM, tal como fue realizado en las pruebas descritas en esta tesina.

En el caso de OpenAI, para comenzar a comunicarse con la API, es necesario tener una cuenta y obtener una *key*, la cual es utilizada cada vez que uno se comunica con la API REST provista. El uso de los distintos modelos expuestos por OpenAI, por ejemplo GPT 3.5 y GPT 4, que varían en la cantidad de parámetros en que fueron entrenados los modelos, además de algunas características como reconocimiento de imágenes, tienen distintos precios y su coste varía según la cantidad de *tokens* utilizados, donde básicamente un *token* se puede pensar como piezas de palabras usados para el lenguaje de procesamiento natural, donde la media de 1 token, en el idioma inglés, equivale a 4 caracteres o 0.75 palabras [24].

Para efectos de la prueba se utilizó el SDK (Kit de Desarrollo de Software, en español) oficial de OpenAI que incluye bibliotecas para facilitar el uso de éste, a diferencia de estar recreando todos los *endpoints* necesarios para su comunicación vía *HTTPS*, y utilizando el lenguaje de programación Python.

Al comunicarse con la API es necesario conocer tres roles, los cuales corresponden a:

- *System*, éste rol provee las instrucciones a alto nivel.
- *User*, éste rol es el que hace las consultas.
- *Assistant*, éste rol corresponde a las respuestas realizadas por el modelo.

En el caso del rol *system*, se definieron los distintos escenarios respecto al alcance que pueda tener las instrucciones y respuestas que recibirá y entregará la API; por ejemplo, el caso principal en esta prueba correspondió a ser un asistente que guía en la migración de contenedores individuales hacia contenedores en un orquestador. Se pueden tener distintos roles de *system* según cada escenario, como es el caso del escenario cuando se está esperando un *Dockerfile*, o cuando se espera por el *output* de un comando, entre otros. Debido a la persistencia que existe en cada mensaje enviado a la API, se pueden



entregar instrucciones de forma constante, como por ejemplo, cómo ordenar y presentar la información. Para el ejemplo, se pidió llevar en forma de tarea, con divisiones y con una puntuación en particular.

Uno de los mensajes como el rol de system, en la etapa posterior de la entrega del *Dockerfile* correspondió a la siguiente instrucción donde se le reitera de forma permanente el rol que debe tomar, hacia quién va dirigido, cual es la meta y como ir mostrando la información:

“Quiero que lideres y optimices tus tareas como Senior DevOps Engineer. Imagina que un DevOps junior está realizando estas operaciones para poder convertir aplicaciones en microservicios que están todas en un servidor Linux con Docker y se quiere migrar a un ambiente de Kubernetes local con minikube. Tu eres el Senior DevOps que debes guiarlo en qué debe realizar en adelante. Debes mantener una "to-do list", que contiene todos los puntos respecto a la implementación y su posterior ejecución de comandos en la terminal. Cada punto interesante debe ser considerado como una tarea que debe ser ejecutada por el junior devops. Las tareas deben estar estructuradas en forma de árbol debido a que una tarea puede ser considerada como una subtarea de otra. Por ejemplo, después de identificar que el Dockerfile utiliza el puerto 80, debes saber que se requiere un Service de Kubernetes para funcionar, por lo que serían dos sub-tareas bajo la prueba de implementación de una aplicación. Puedes mostrar las tareas como una estructura de capas, por ejemplo 1, 1.1, 1.1.1, etc.

Cada vez que recibes una actualización (puedes imaginar que esta información te la pasa el junior devops engineer), tu debes:

- 1. Decidir eliminar algunas tareas si están consideradas como completadas.*
- 2. Decidir agregar nuevas tareas si hay algo interesante.*
- 3. Darle puntaje a cada subtarea, mostrando si puede llegar a una posible complicación de implementación.*
- 4. Agregar una línea divisora (-----). Luego recomendar que hacer de forma siguiente basada en los puntajes cuando te pregunte acerca del to-do, listadas en una secuencia como 1, 2, 3,*

Debes recordar que las tareas debes mantenerlas de forma clara, precisas y cortas por el límite de tamaño de los token. Debes recordar de eliminar tareas redundantes/outdated por la misma razón.

Yo voy a proveer la información general de conversión en corto. "

A diferencia de una aplicación tradicional donde todos las instrucciones descritas anteriormente tendría que haber sido programado específicamente, por ejemplo, creando un sistema de consideración de tareas según importancia, la interfaz para crear una tarea y sus subtareas, entre otros, estas instrucciones son dirigidas al LLM con lenguaje natural y que, debido a su entrenamiento, logra interpretar como instrucciones específicas.

La aplicación se programó para poder leer archivos, en este caso los distintos *Dockerfile*, y poder volcarlos hacia OpenAI, a diferencia de tener que estar copiando y pegando cada archivo *Dockerfile*, junto a las instrucciones entregadas descritas anteriormente. Los resultados obtenidos corresponden al resultado entregado por la API, tomando el rol *assistant*, incorporando las instrucciones descritas, rol *system*, junto al *Dockerfile* entregado, rol *user* (ver Anexos 44 y 45).

Finalmente, debido a que la instrucción general incorpora el hacer que el usuario ejecute los comandos junto con una guía, además de la definición de tareas y subtareas, y luego entregue el output para su análisis, como por ejemplo, ver si existe algún error o si el microservicio se desplegó correctamente, entregó una mayor fluidez y facilidad respecto a ir completando la tarea, referente a la migración de contenedores individuales hacia contenedores orquestados por Kubernetes, lo que permite tener un acercamiento a un asistente guiando de forma personalizada.

5.7 Trabajo futuro

Los Large Language Models (LLM) son modelos generalistas entrenados con una amplia variedad de información. Sin embargo, es posible especializarlos mediante *fine-tuning*, entrenándolos con datos específicos del dominio de interés. En el contexto de esta investigación, se podría realizar un *fine-tuning* de un modelo de OpenAI utilizando documentación sobre contenedores, Kubernetes, Docker, arquitectura de microservicios, etc. Esto mejoraría la capacidad del modelo para responder a preguntas y generar soluciones precisas en este ámbito, incluyendo el uso de las últimas versiones de herramientas y la identificación de buenas prácticas de seguridad para Kubernetes.

Otro aspecto a explorar es el uso exhaustivo de la API de OpenAI para automatizar la interacción con el clúster de Kubernetes. Una versión mejorada de la aplicación desarrollada podría consultar y ejecutar acciones en el clúster basándose en las respuestas del modelo, e incluso intentar solucionar errores de forma autónoma. Este enfoque permitiría reducir la intervención manual y aumentar la eficiencia en la gestión de Kubernetes, especialmente en entornos de prueba e investigación.

5.8 Resultado de la Investigación

La hipótesis de esta investigación postulaba que el uso de Large Language Models (LLM) permitiría una migración exitosa de microservicios desde contenedores individuales en Docker hacia un entorno orquestado con Kubernetes, generando configuraciones, documentación y pruebas necesarias, sin necesidad de incorporar nuevos roles en la empresa y con un tiempo de implementación reducido. Los resultados obtenidos a través de diversas pruebas respaldan esta hipótesis de manera parcial.

En primer lugar, las pruebas de funcionalidad demostraron que los LLM, tanto a través de chatbots como mediante una aplicación dedicada con acceso a la API, fueron capaces de generar configuraciones válidas para la mayoría de los microservicios, permitiendo su ejecución y comunicación en el entorno de Kubernetes. Esto se alinea con la hipótesis al demostrar que los LLM pueden automatizar y simplificar la generación de configuraciones, reduciendo la necesidad de conocimientos especializados en Kubernetes.

Sin embargo, no todos los microservicios funcionaron a la primera, ya que algunos requerían parámetros adicionales, como variables de entorno, que no fueron proporcionados inicialmente por el LLM. Esto indica que, si bien los LLM pueden generar configuraciones básicas, aún se requiere supervisión y ajustes manuales para asegurar el funcionamiento completo de aplicaciones más complejas.

En cuanto a las pruebas de escalabilidad, los resultados fueron mixtos. Aunque el LLM pudo proporcionar sugerencias y configuraciones para el autoescalado de los microservicios, no se logró un escalamiento exitoso bajo una carga de prueba. Esto sugiere que, si bien los LLM pueden ayudar en la configuración del autoescalado, su efectividad puede depender de la complejidad de la aplicación y de la carga de trabajo específica. Además, la información proporcionada por el LLM sobre el autoescalado no siempre estuvo actualizada con las últimas versiones de Kubernetes, lo que resalta la importancia de validar la información proporcionada por los LLM con fuentes oficiales y actualizadas.

Respecto a la seguridad, las pruebas revelaron que las configuraciones generadas por el LLM no siempre cumplían con las mejores prácticas de seguridad de Kubernetes. Se identificaron configuraciones inseguras en el plano de control y falta de políticas de red en los microservicios, lo que podría exponer el clúster a vulnerabilidades. Aunque el LLM pudo proporcionar soluciones para algunos de estos problemas, no logró mitigar todas las vulnerabilidades encontradas. Esto indica que, si bien los LLM pueden ayudar en la identificación y mitigación de problemas de seguridad, es fundamental contar con expertos en seguridad que supervisen y validen las configuraciones generadas por los LLM.

En cuanto a la documentación, el LLM demostró ser muy efectivo en la generación de documentación clara y concisa sobre las configuraciones de Kubernetes y su justificación. Esto respalda la hipótesis al demostrar que los LLM pueden automatizar la generación de documentación, facilitando la comprensión y el mantenimiento del entorno de Kubernetes.

En comparación con la migración realizada sin LLM, se observó que, aunque se requirió más tiempo y conocimientos especializados en Kubernetes, se logró un despliegue funcional de los microservicios. Esto sugiere que los LLM pueden acelerar significativamente el proceso de migración, pero no reemplazan por completo la necesidad de conocimientos especializados, especialmente en áreas como seguridad y optimización del rendimiento.

6. Conclusiones

A partir de la investigación realizada, se concluye que los Large Language Models (LLM) públicos, como ChatGPT, pueden acelerar significativamente la adopción de tecnologías complejas como Kubernetes. Al proporcionar información clara, ejemplos relevantes y respuestas personalizadas, los LLM facilitan la comprensión y aplicación de conceptos técnicos, reduciendo la curva de aprendizaje. Sin embargo, es crucial tener en cuenta que los LLM son modelos generalistas y pueden requerir supervisión experta para validar y ajustar las soluciones propuestas, especialmente en contextos específicos o con configuraciones personalizadas.

La integración de APIs de LLM en aplicaciones empresariales, como se demostró con el desarrollo de una herramienta de migración personalizada, ofrece un mayor control y personalización, mejorando la experiencia del usuario y la eficiencia en la adopción de nuevas tecnologías. A pesar de las ventajas, es fundamental considerar la necesidad de supervisión humana y la implementación de medidas de seguridad para mitigar posibles errores o vulnerabilidades en las soluciones generadas por los LLM.

El *prompt engineering*, es decir, la capacidad de formular preguntas e instrucciones de manera efectiva, es esencial para obtener resultados precisos y útiles de los LLM. Dominar esta técnica permite aprovechar al máximo el potencial de los LLM y mejorar la calidad de las soluciones generadas.

En resumen, los LLM representan una herramienta prometedora para acelerar la adopción de nuevas tecnologías y mejorar la eficiencia en entornos empresariales. Sin embargo, su uso debe ser complementado con supervisión experta y un enfoque cuidadoso en la seguridad y la validación de las soluciones propuestas.

7. Referencias

- [1] Dynatrace (16/01/2023). “Kubernetes in the wild report 2023”. Disponible en: <https://www.dynatrace.com/news/blog/kubernetes-in-the-wild-2023/>
- [2] WX Zhao, et al. A Survey of Large Language Models. 2023
- [3] A Vaswani, et al. Attention Is All You Need. 2017
- [4] Radford, et al. Improving Language Understanding by Generative Pre-Training. 2018
- [5] OpenAI (13/03/2023). “GPT-4”. Disponible en: <https://openai.com/gpt-4>
- [6] R. Thoppilan, D. De Freitas, et al. LaMDA: Language Models for Dialog Applications. 2022.
- [7] D. Bernstein, "Containers and Cloud: From LXC to Docker to Kubernetes," in IEEE Cloud Computing, vol. 1, no. 3, pp. 81-84, Sept. 2014, doi: 10.1109/MCC.2014.51.
- [8] C. Pahl, "Containerization and the PaaS Cloud," in IEEE Cloud Computing, vol. 2, no. 3, pp. 24-31, May-June 2015, doi: 10.1109/MCC.2015.51.
- [9] Datadog (01/10/2021). “10 Trends in Real-World Container Use”. Disponible en: <https://www.datadoghq.com/container-report-2021/>
- [10] Kubernetes (17/07/2022). “¿Qué es Kubernetes?”. Disponible en: <https://kubernetes.io/es/docs/concepts/overview/what-is-kubernetes/>



- [11] G. Daniel and J. Cabot, "The Software Challenges of Building Smart Chatbots," 2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), Madrid, ES, 2021, pp. 324-325, doi: 10.1109/ICSE-Companion52605.2021.00138.
- [12] Chen, B., Zhang, Z., Langrené, N., & Zhu, S. (2023). Unleashing the potential of prompt engineering in Large Language Models: a comprehensive review. arXiv preprint arXiv:2310.14735.
- [13] D. Sobania, M. Briesch, C. Hanna and J. Petke, "An Analysis of the Automatic Bug Fixing Performance of ChatGPT," 2023 IEEE/ACM International Workshop on Automated Program Repair (APR), Melbourne, Australia, 2023, pp. 23-30, doi: 10.1109/APR59189.2023.00012.
- [14] M. Gupta, C. Akiri, K. Aryal, E. Parker and L. Praharaj, "From ChatGPT to ThreatGPT: Impact of Generative AI in Cybersecurity and Privacy," in IEEE Access, vol. 11, pp. 80218-80245, 2023, doi: 10.1109/ACCESS.2023.3300381.
- [15] GitHub. "Sample cloud-first application with 10 microservices showcasing Kubernetes, Istio, and gRPC.". Disponible en: <https://github.com/GoogleCloudPlatform/microservices-demo>
- [16] NSA-CISA (29/08/2022). "Kubernetes Hardening Guide". Disponible en: https://media.defense.gov/2022/Aug/29/2003066362/-1/-1/0/CTR_KUBERNETES_HARDENING_GUIDANCE_1.2_20220829.PDF
- [17] MITRE ATT&CK. "Matrix - Enterprise". Disponible en: <https://attack.mitre.org/matrices/enterprise/containers/>
- [18] Center for Internet Security. "CIS Kubernetes Benchmark". Disponible en: <https://www.cisecurity.org/benchmark/kubernetes>
- [19] Minikube. "minikube quickly sets up a local Kubernetes cluster on macOS, Linux, and Windows.". Disponible en: <https://minikube.sigs.k8s.io/docs/>
- [20] Google. "Google Kubernetes Engine". Disponible en <https://cloud.google.com/kubernetes-engine>
- [21] Kubescape. "An open-source Kubernetes security platform for your IDE, CI/CD pipelines, and clusters". Disponible en: <https://kubescape.io/>
- [22] Docker. "Docker Compose overview". Disponible en <https://docs.docker.com/compose/>
- [23] Kompose. "Convert your Docker Compose file to Kubernetes". Disponible en <https://kompose.io/>
- [24] OpenAI. "Pricing". Disponible en: <https://openai.com/pricing>

Anexos



ChatGPT

¡Entendido! Para comenzar, necesitare información sobre el primer microservicio que deseas convertir a Kubernetes. Proporcióname el Dockerfile del microservicio y cualquier otra información relevante sobre su configuración y dependencias.

Una vez que tengamos esa información, podré proporcionarte los comandos necesarios para crear la imagen de Docker, subirla a un registro (como Docker Hub) y los archivos de configuración para desplegar el microservicio en Kubernetes.

Cuando estés listo con la información del primer microservicio, puedes proporcionarme el Dockerfile y cualquier detalle adicional.

Anexo 1. Respuesta de ChatGPT al prompt principal.

 Tú
este es el primer servicio, se llama adservice:

```
# Copyright 2020 Google LLC
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

FROM eclipse-temurin:19@sha256:f3bf1ad599d4b5dbdd7ceb55708d10cb9fafb08e094ef91e92aa63b520a232e
as builder

WORKDIR /app

COPY ["build.gradle", "gradlew", "."]
COPY gradle gradle
RUN chmod +x gradlew
RUN ./gradlew downloadRepos

COPY . .
RUN chmod +x gradlew
RUN ./gradlew installDist

FROM eclipse-temurin:19.0.1_10-jre-alpine@sha256:fabe27bd9db502d484a11c771c2f4ef7bba4a172527084d939935358fb06c4
```

Anexo 2. Entrega de contenido de Dockerfile a ChatGPT.

 **ChatGPT**

Perfecto, gracias por proporcionar el Dockerfile del microservicio "adservice". Ahora, procederemos a los siguientes pasos:

Paso 1: Crear la imagen de Docker

Utilizaremos el Dockerfile proporcionado para construir la imagen del microservicio "adservice". Puedes ejecutar estos comandos en el directorio donde se encuentra el Dockerfile:

```
bash Copy code  
  
# Construir la imagen  
docker build -t adservice:latest .  
  
# Verificar que la imagen se haya creado correctamente  
docker images
```

Anexo 3. Respuesta con contenido de Dockerfile entregado - Paso 1.

Paso 2: Subir la imagen a un registro (por ejemplo, Docker Hub)

Si aún no tienes una cuenta en Docker Hub, puedes crear una en <https://hub.docker.com/>. Luego, sigue estos pasos:

```
bash Copy code  
  
# Inicia sesión en Docker Hub (ingresa tus credenciales cuando se te solicite)  
docker login  
  
# Etiqueta la imagen con tu nombre de usuario de Docker Hub (reemplaza 'tu-usuario')  
docker tag adservice:latest tu-usuario/adservice:latest  
  
# Sube la imagen al registro  
docker push tu-usuario/adservice:latest
```

Anexo 4. Respuesta con contenido de Dockerfile entregado - Paso 2.

Paso 3: Desplegar en Kubernetes

A continuación, proporcionaré los archivos de configuración YAML para desplegar el microservicio en Kubernetes. Puedes crear estos archivos en tu máquina local:

adservice-deployment.yaml:

yaml

Copy code

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: adservice
spec:
  replicas: 3 # ajusta según sea necesario
  selector:
    matchLabels:
      app: adservice
  template:
    metadata:
      labels:
        app: adservice
    spec:
      containers:
        - name: adservice
          image: tu-usuario/adservice:latest
          ports:
            - containerPort: 9555
```

Anexo 5. Respuesta con contenido de Dockerfile entregado - Paso 3.

Paso 4: Aplicar la configuración en Kubernetes

Ejecuta los siguientes comandos en tu terminal para desplegar el microservicio en Kubernetes:

bash

Copy code

```
# Aplicar la configuración
kubectl apply -f adservice-deployment.yaml
kubectl apply -f adservice-service.yaml
```

Esperaré a que completes estos pasos. Cuando estés listo, revisa el estado del despliegue ejecutando ``kubectl get pods`` y proporciona el output para continuar con el análisis.

Anexo 6. Respuesta con contenido de Dockerfile entregado - Paso 4.

**Tú**

```
fsaez@xpsizard:~/usm/2023/microservices-demo-0.8.0/sandbox$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
adservice-796df94b99-5lbdw	1/1	Running	0	38s
adservice-796df94b99-j2r8r	1/1	Running	0	38s
adservice-796df94b99-kxn49	1/1	Running	0	38s

**ChatGPT**

Excelente, parece que el despliegue del microservicio "adservice" en Kubernetes fue exitoso. Todos los pods están en estado "Running", lo cual es un buen indicador.

Para verificar aún más, puedes ejecutar el siguiente comando para obtener información adicional sobre los servicios y su estado:

```
bash
```

```
kubectl get services
```

 Copy code

Este comando mostrará la información sobre los servicios, incluido el servicio "adservice". Asegúrate de que el servicio tenga una dirección IP externa asignada (especialmente si estás utilizando el tipo de servicio LoadBalancer) y que esté en un estado "Running".

Luego, puedes probar el servicio ejecutando una solicitud. Si el servicio "adservice" expone una API, puedes hacer una solicitud HTTP a uno de los pods. Si no estás seguro de la interfaz expuesta, házmelo saber, y te proporcionaré un ejemplo de solicitud.

Además, si hay otros microservicios que deseas desplegar o si tienes alguna pregunta adicional, házmelo saber. ¡Estoy aquí para ayudar!

Anexo 7. Respuesta considerando el estado de los servicios.

**Tú**

NAME	READY	STATUS	RESTARTS	AGE
adservice-796df94b99-kxn49	1/1	Running	0	17m
cartservice-7cf97f4966-xgbmz	1/1	Running	0	8m37s
checkoutservice-8595fcc57b-bh97v	0/1	CrashLoopBackOff	2 (20s ago)	42s

**ChatGPT**

Parece que el pod del servicio "checkoutservice" está experimentando problemas y se encuentra en estado `CrashLoopBackOff`, lo que indica que el contenedor no está iniciando correctamente.

Vamos a verificar los registros del pod para obtener más información sobre el error. Puedes hacerlo usando el siguiente comando:

```
bash
```

```
kubectl logs checkoutservice-8595fcc57b-bh97v
```

Este comando te dará más detalles sobre lo que está sucediendo dentro del contenedor y por qué está fallando. Por favor, proporciona la salida de esos registros para que pueda ayudarte a identificar y solucionar el problema.

Si la salida de los registros no es clara, puedes copiarla aquí, y trataré de ayudarte a entender la causa del fallo.

Anexo 8. Problemas con pod e intento de solución

**Tú**

595fcc57b-bh97v
{\"message\": \"Tracing disabled.\", \"severity\": \"info\", \"timestamp\": \"2023-12-19T04:53:54.660821657Z\"}
{\"message\": \"Profiling disabled.\", \"severity\": \"info\", \"timestamp\": \"2023-12-19T04:53:54.661202209Z\"}
panic: environment variable \"SHIPPING_SERVICE_ADDR\" not set

goroutine 1 [running]:
main.mustMapEnv(0xc0000980d8, {0xb931d6, 0x15})
 /src/main.go:201 +0xb9
main.main()
 /src/main.go:107 +0x21c

**ChatGPT**

El error indica que hay una variable de entorno faltante, en este caso, `SHIPPING\_SERVICE\_ADDR`, que el servicio \"checkoutservice\" espera pero no se ha configurado.

Para solucionar esto, necesitas asegurarte de que todas las variables de entorno necesarias estén disponibles al iniciar el contenedor del servicio \"checkoutservice\".

Aquí hay algunos pasos que puedes seguir:

1. **Actualizar el Dockerfile:** Agrega las variables de entorno necesarias directamente en el Dockerfile. Por ejemplo, al final del Dockerfile, puedes incluir líneas como estas:

dockerfile

Copy code

```
ENV SHIPPING_SERVICE_ADDR=localhost:8080  
# Agrega otras variables de entorno si es necesario
```

2. **Reconstruir y Publicar la Imagen:** Después de hacer cambios en el Dockerfile, reconstruye la imagen

Anexo 9. Problemas con pod e intento de solución

**Tú**

NAME	READY	STATUS	RESTARTS	AGE
adservice-796df94b99-kxn49	1/1	Running	0	39m
cartservice-7cf97f4966-xgbmz	1/1	Running	0	30m
checkoutservice-594fd67ff6-6frh2	1/1	Running	0	5m49s
currencyservice-5cc6bd79c6-rvkvv	1/1	Running	0	3m40s
paymentservice-6fb8df6599-j89c5	1/1	Running	0	27s
productcatalogservice-799697dcc9-r7mhh	1/1	Running	0	13m
shippingservice-7d69fbc459-s8gfb	1/1	Running	0	18m

**ChatGPT**

¡Excelente! Parece que el servicio \"paymentservice\" se ha desplegado correctamente y está en funcionamiento. ¿Hay algún otro microservicio que te gustaría convertir y desplegar en Kubernetes a través de Minikube? O ¿hay algo más en lo que pueda ayudarte?

Anexo 10. Evolución en el contexto.

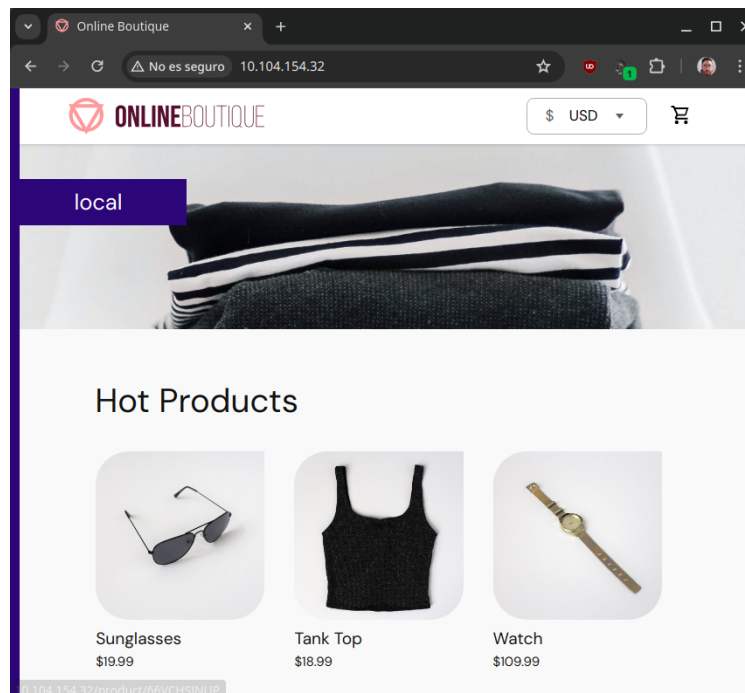
29

Services(default)[12]				
NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORTS
adservice	LoadBalancer	10.107.18.119	10.107.18.119	80>32246
cartservice	LoadBalancer	10.105.14.177	10.105.14.177	80>31422
checkoutservice	LoadBalancer	10.105.112.29	10.105.112.29	80>30505
currencyservice	LoadBalancer	10.104.71.55	10.104.71.55	80>31700
emailservice	LoadBalancer	10.99.226.50	10.99.226.50	80>31963
frontend	LoadBalancer	10.104.154.32	10.104.154.32	80>31989
kubernetes	ClusterIP	10.96.0.1		https:443>0
loadgenerator	LoadBalancer	10.96.110.19	10.96.110.19	80>32197
paymentservice	LoadBalancer	10.100.171.124	10.100.171.124	80>30442
productcatalogservice	LoadBalancer	10.96.246.95	10.96.246.95	80>31891
recommendationservice	LoadBalancer	10.98.199.239	10.98.199.239	80>32356
shippingservice	LoadBalancer	10.109.98.196	10.109.98.196	80>31662

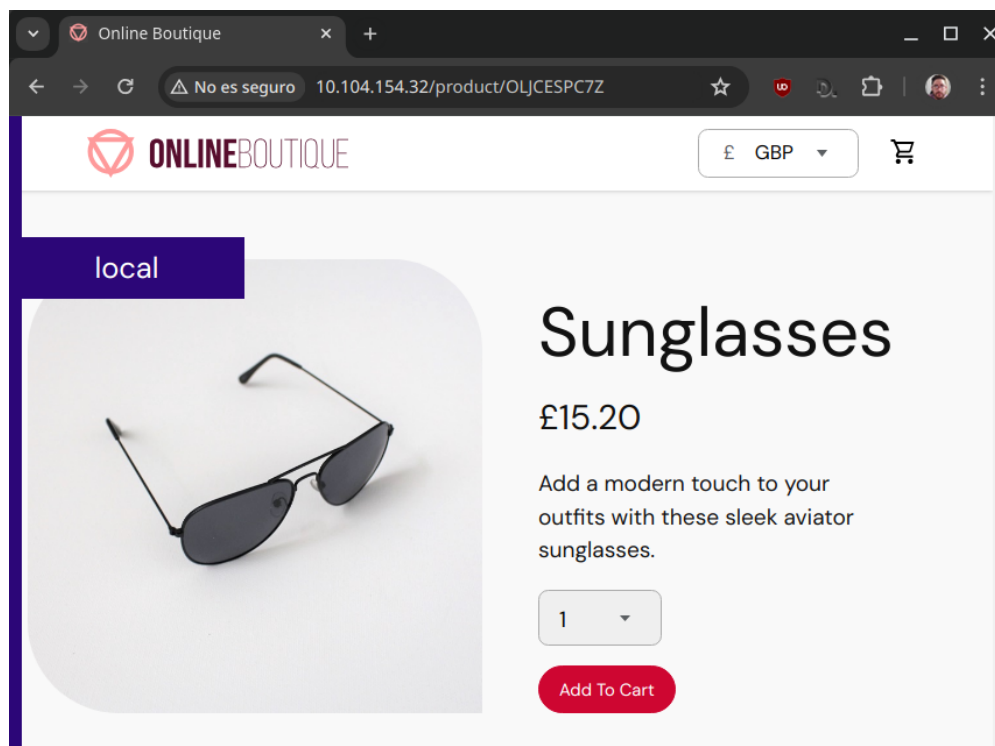
Anexo 11. Listado de direcciones IP del cluster.

```
/tmp $ hostname
cartservice-7cf97f4966-xgbmz
/tmp $ ./grpcurl -plaintext -d '{"context_keys": ["key1", "key2"]}' -proto demo.proto
adservice:80 hipstershop.AdService/GetAds
{
  "ads": [
    {
      "redirectUrl": "/product/L9ECAV7KIM",
      "text": "Loafers for sale. Buy one, get second one for free"
    },
    {
      "redirectUrl": "/product/66VCHSJNUP",
      "text": "Tank top for sale. 20% off."
    }
  ]
}
```

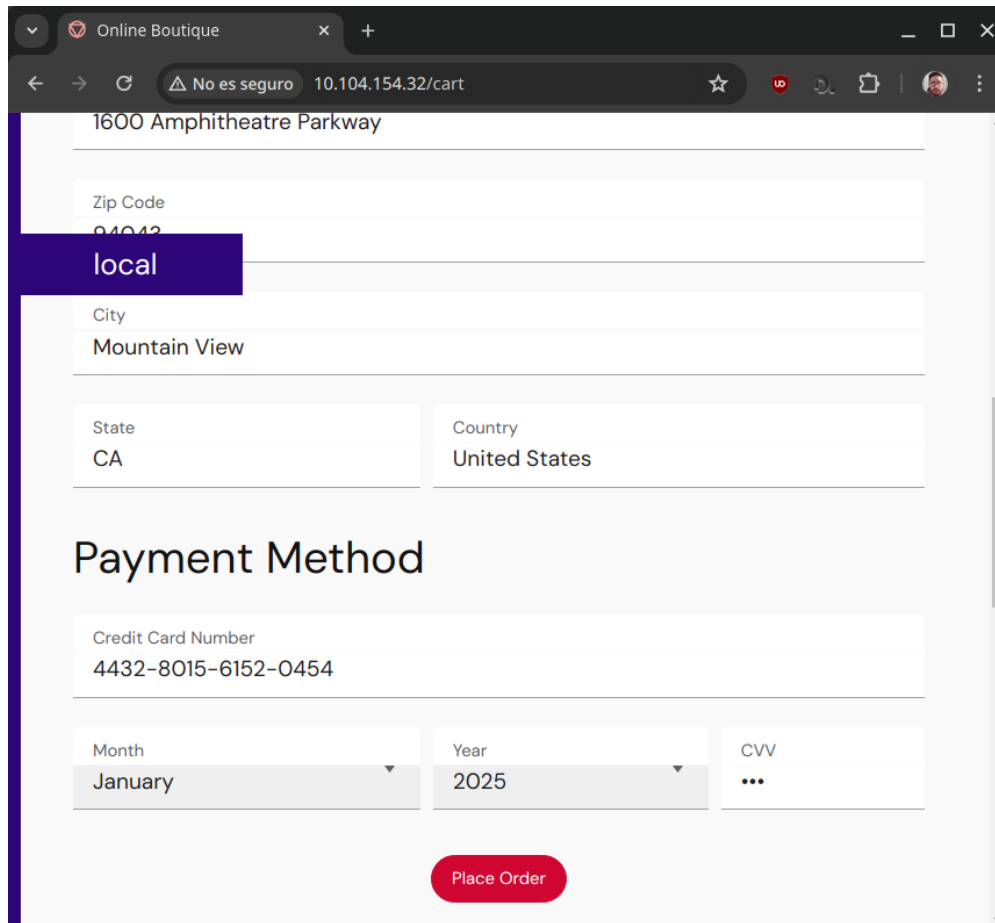
Anexo 12. Prueba de conectividad desde un microservicio a otro



Anexo 13. Captura de inicio de página web.



Anexo 14. Captura web de la página mostrando un objeto junto a una conversión distinta.



Anexo 15. Captura web donde se indican los datos para proveer orden.

```
Logs(default/emailservice-5f757cdb8c-5cdqb:emailservice)[5m]
Autoscroll:On FullScreen:Off Timestamps:Off Wrap:On

{"timestamp": 1713151216.2844336, "severity": "INFO", "name": "emailservice-server", "message": "A request to send order confirmation email to someone@example.com has been received."}
```

Anexo 16. Registro del servicio de correo donde indica la simulación del envío.

```

Logs(default/payment-service-6fb8df6599-j89c5.payment-service)[5m]
Autoscroll:On      FullScreen:Off      Timestamps:Off      Wrap:On

{"severity":"info","time":1713151216281,"pid":1,"hostname":"payment-service-6fb8df6599-j89c5","name":"payment-service-server","message":"PaymentService#Charge invoked with request {\"amount\":{\"currency_code\":\"GBP\",\"units\":\"22\",\"nanos\":38130030},\"credit_card\":{\"credit_card_number\":\"4432-8015-6152-0454\",\"credit_card_cvv\":672,\"credit_card_expiration_year\":2025,\"credit_card_expiration_month\":1}}"}
{"severity":"info","time":1713151216281,"pid":1,"hostname":"payment-service-6fb8df6599-j89c5","name":"payment-service-charge","message":"Transaction processed: visa ending 0454 Amount: GBP22.38130030"}

```

Anexo 17. Información referente al registro del servicio de pagos.

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	3817	0(0.00%)	9551	277	39117	5600	64.70	0.00
GET	/cart	1663	0(0.00%)	2088	248	4788	2000	51.30	0.00
POST	/cart	1472	0(0.00%)	3286	709	5936	3600	40.30	0.00
POST	/cart/checkout	430	0(0.00%)	2261	322	4796	2200	15.60	0.00
GET	/product/0PUK6V6EV0	809	0(0.00%)	2064	290	4837	2000	23.70	0.00
GET	/product/1YMW1N1U40	806	0(0.00%)	2015	284	4825	2000	23.20	0.00
GET	/product/2ZYFJ3GW2N	809	0(0.00%)	2082	288	4838	2000	24.70	0.00
GET	/product/66VCHSJMUP	824	0(0.00%)	2118	296	4832	2100	26.00	0.00
GET	/product/6E92ZHYVYZ	800	0(0.00%)	2107	316	4740	2000	24.70	0.00
GET	/product/9SIQT8T0J0	819	0(0.00%)	2080	267	4605	2100	25.50	0.00
GET	/product/L9ECAV7KIM	815	0(0.00%)	2110	339	4832	2000	24.80	0.00
GET	/product/LS4PSXUNUM	786	0(0.00%)	2064	284	4700	2000	21.70	0.00
GET	/product/OLJCESPC7Z	829	0(0.00%)	2082	311	4774	2000	25.50	0.00
POST	/setCurrency	1167	0(0.00%)	2439	514	4795	2400	44.70	0.00
Aggregated		15846	0(0.00%)	4024	248	39117	2400	436.40	0.00

Anexo 18. Información entregada por aplicación locust al inicio de su ejecución.

NAME	PF	READY	RESTARTS	STATUS	CPU	MEM
ad-service-796df94b99-sbr77	●	1/1	1	Running	108	83
cart-service-7cf97f4966-ncq6g	●	1/1	1	Running	30	15
checkout-service-6f87cb4b4d-7st7v	●	1/1	1	Running	2	6
currency-service-5cc6bd79c6-2qn5h	●	1/1	1	Running	14	15
email-service-5f757cdb8c-jdxj2	●	1/1	1	Running	16	39
frontend-6455789f65-c2snc	●	1/1	1	Running	2	6
load-generator-6db5967845-6zvxf	●	1/1	0	Running	0	0
load-generator-6db5967845-kqrk5	●	1/1	0	Running	0	0
load-generator-6db5967845-n47kz	●	1/1	0	Running	0	0
load-generator-6db5967845-rwpt2	●	1/1	0	Running	0	0
load-generator-6db5967845-wwl6z	●	1/1	0	Running	0	0
payment-service-6fb8df6599-j89c5	●	1/1	3	Running	12	15
product-catalog-service-799697dcc9-r7mhh	●	1/1	3	Running	2	12
recommendations-service-6859dcdc-zdztm	●	1/1	3	Running	25	51
shipping-service-7d69fbc459-s8gfb	●	1/1	3	Running	1	8

Anexo 19. Uso de CPU y memoria de los contenedores al inicio del ejercicio de generación de carga.

		Autoscroll:On		FullScreen:Off		Timestamps:Off		Wrap:Off	
GET	/product/6E92ZMYFZ	1	0(0.00%)	29031	29031	29031	29031		
GET	/product/LS4PSXUNUM	1	0(0.00%)	29001	29001	29001	29001		
Aggregated		263	260(98.86%)	54919	29001	80184	60000		
Type	Name	# reqs	# fails	Avg	Min	Max	Med		
GET	/	260	260(100.00%)	55210	31216	80184	60000		
GET	/cart	2	0(0.00%)	75903	75899	75908	75908		
GET	/product/0PUK6V6EV0	1	0(0.00%)	30990	30990	30990	30990		
GET	/product/6E92ZMYFZ	1	0(0.00%)	29031	29031	29031	29031		
GET	/product/LS4PSXUNUM	2	0(0.00%)	52108	29001	75215	29001		
GET	/product/OLJCESPC7Z	1	0(0.00%)	75123	75123	75123	75123		
Aggregated		267	260(97.38%)	55228	29001	80184	60000		

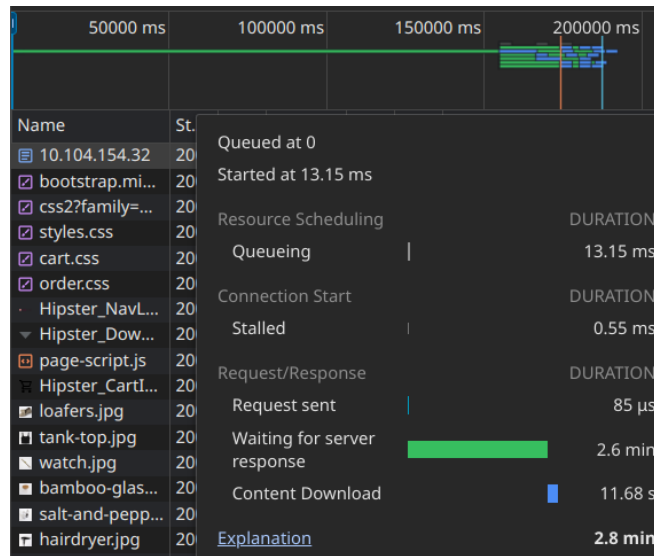
Anexo 20. Información entregada por aplicación locust después de un tiempo siendo ejecutado.

NAME	PF	READY	RESTARTS	STATUS	CPU	MEM
adservice-796df94b99-sbr77	●	1/1	1	Running	24	179
cartservice-7cf97f4966-ncq6g	●	1/1	1	Running	38	28
checkoutservice-6f87cb4b4d-7st7v	●	1/1	1	Running	5	9
currencyservice-5cc6bd79c6-2qn5h	●	1/1	1	Running	358	1448
emailservice-5f757cdb8c-jdxj2	●	1/1	1	Running	2	18
frontend-6455789f65-c2snc	●	1/1	1	Running	392	6945
loadgenerator-6db5967845-6zvxv	●	1/1	0	Running	401	1323
loadgenerator-6db5967845-kqrk5	●	1/1	0	Running	410	1399
loadgenerator-6db5967845-n47kz	●	1/1	0	Running	425	1370
loadgenerator-6db5967845-rwpt2	●	1/1	0	Running	403	1389
loadgenerator-6db5967845-wwl6z	●	1/1	0	Running	382	1365
paymentservice-6fb8df6599-j89c5	●	1/1	3	Running	3	16
productcatalogservice-799697dcc9-r7mhh	●	1/1	3	Running	96	12
recommendationservice-6859dcdc-zdztm	●	1/1	3	Running	131	55
shippingservice-7d69fbc459-s8gfb	●	1/1	3	Running	7	13

Anexo 21. Uso de CPU y memoria de los contenedores al intermedio del ejercicio de generación de carga.

Logs(default/frontend-6455789f65-c2snc:frontend)[1m]									
Autoscroll:Off		FullScreen:Off		Timestamps:Off		Wrap:Off			
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "51b8230c-2b8c-446d-90"									
{ "http.req.id": "018cdf23-1617-4220-bc8a-0ae34dc696bc", "http.req.method": "POST", "http.req.path": "/cart", "http.res.bytes": 0, "http.res.s									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "56850217-b825-408d-ba"									
{ "http.req.id": "fc5ca7c2-202a-4e23-bb98-13009fe2b729", "http.req.method": "GET", "http.req.path": "/", "http.res.bytes": 10216, "http.res.s									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "e55b5e82-412b-4b71-b8"									
{ "http.req.id": "38d52c63-7510-4903-ad50-dba9c5662293", "http.req.method": "GET", "http.req.path": "/", "http.res.bytes": 10216, "http.res.s									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "f32ea5c9-ffe4-4639-ab"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "4dddec16-d6a9-4bae-89"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "bf7cbb14-9009-46aa-b8"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "47e75212-dbf0-4a86-95"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "e3e058ae-6458-4dd7-ac"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "9efeab62-0ab1-42e1-9c"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "32127303-e212-4804-bc"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "fa115f81-09d5-40a3-b3"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "52661f43-7f47-4e4c-b4"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "4747d868-2578-4d8c-ba"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "9176a3f5-88e3-4daf-9b"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "93bb6cab-d300-41b1-9f"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "a4a8ebad-813b-4025-8b"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "ed938c40-3067-4a95-af"									
{ "error": "failed to get ads: rpc error: code = DeadlineExceeded desc = context deadline exceeded", "http.req.id": "89f58cf3-1ad8-413b-90"									

Anexo 22. Registro de microservicio frontend reportando errores de comunicación.



Anexo 23. Captura de Chrome respecto a la demora en el tiempo de respuesta del servidor.

NAME	PF	READY	RESTARTS	STATUS	CPU	MEM
adservice-796df94b99-sbr77	●	1/1	1	Running	9	32
cartservice-7cf97f4966-ncq6g	●	1/1	1	Running	7	32
checkoutservice-6f87cb4b4d-7st7v	●	1/1	1	Running	1	9
currencyservice-5cc6bd79c6-2qn5h	●	1/1	1	Running	81	1092
emailservice-5f757cdb8c-jdxj2	●	1/1	1	Running	1	10
frontend-6455789f65-c2snc	●	1/1	1	Running	523	9411
paymentservice-6fb8df6599-j89c5	●	1/1	3	Running	1	15
productcatalogservice-799697dcc9-r7mhh	●	1/1	3	Running	10	14
recommendationservice-6859dcde-zdztm	●	1/1	3	Running	36	48
shippingservice-7d69fbc459-s8gfb	●	1/1	3	Running	1	10

Anexo 24. Uso de CPU y memoria de los contenedores después del ejercicio de generación de carga.

 **Tú**
Actualmente los microservicios de frontend y currency no están escalando. ¿de qué manera se puede lograr escalar éstos en Kubernetes de manera automática?

 **ChatGPT**
Para lograr el escalado automático de los microservicios de frontend y currency en Kubernetes, puedes utilizar los objetos `HorizontalPodAutoscaler` (HPA). El HPA ajustará automáticamente el número de réplicas de tus microservicios en función de la carga de trabajo, lo que garantiza que siempre haya suficientes instancias disponibles para manejar la demanda.

Aquí hay un ejemplo de cómo puedes configurar un HPA para cada uno de tus despliegues:

1. **Crear un HorizontalPodAutoscaler para el microservicio de frontend:**

```
yaml Copy code  
  
apiVersion: autoscaling/v2beta2  
kind: HorizontalPodAutoscaler  
metadata:  
  name: frontend-autoscaler  
spec:  
  scaleTargetRef:  
    apiVersion: apps/v1  
    kind: Deployment  
    name: frontend  
  minReplicas: 1  
  maxReplicas: 10  
  metrics:  
  - type: Resource  
    resource:  
      name: cpu  
      targetAverageUtilization: ↓ # Ajusta según tus necesidades
```

Anexo 25. Solución para auto escalar contenedores.

```
Events:  
Type      Reason              Age   From              Message  
-----  
Warning   FailedGetResourceMetric 10s   horizontal-pod-autoscaler failed to get cpu utilization: missing request for cpu in container frontend of Pod frontend-6455789f65-c2snc  
Warning   FailedComputeMetricsReplicas 10s   horizontal-pod-autoscaler invalid metrics (1 invalid out of 1), first error is: failed to get cpu resource metric value: failed to get cpu utilization: missing request for cpu in container frontend of Pod frontend-6455789f65-c2snc
```

Anexo 26. Error en la ejecución de HorizontalPodAutoscaler.

```
- name: AD_SERVICE_ADDR
  value: "adservice:80"
resources:
  requests:
    cpu: "100m" # Solicitar 100 milicore (0.1 core) de CPU
    memory: "256Mi"
  limits:
    cpu: "500m" # Limitar a 500 milicore (0.5 core) de CPU
    memory: "512Mi"
```

En este ejemplo, he agregado la sección `resources` debajo de la definición del contenedor, donde se especifican las solicitudes y los límites de recursos para CPU y memoria. Ajusta los valores de acuerdo a tus necesidades y los recursos disponibles en tu clúster. Una vez que hayas actualizado tu archivo Deployment, puedes volver a aplicarlo para que los cambios surtan efecto.

Anexo 27. Respuesta referente a los parámetros de requests y limits.

NAMESPACE↑	NAME	PF	READY	RESTARTS	STATUS	CPU	MEM	%CPU/R	%CPU/L	%MEM/R
default	adservice-796df94b99-sbr77	●	1/1	1	Running	59	199	n/a	n/a	n/a
default	cartservice-7cf97f4966-ncq6g	●	1/1	1	Running	60	49	n/a	n/a	n/a
default	checkoutservice-6f87cb4b4d-7st7v	●	1/1	1	Running	5	14	n/a	n/a	n/a
default	currencyservice-5574946d56-kzxmt	●	1/1	0	Running	0	0	0	0	0
default	currencyservice-5574946d56-l4zxl	●	1/1	0	Running	0	0	0	0	0
default	currencyservice-5574946d56-mccw2	●	1/1	0	Running	368	87	368	73	34
default	currencyservice-5574946d56-pc44q	●	1/1	0	Running	1	15	1	0	6
default	currencyservice-5574946d56-rn8pw	●	1/1	0	Running	18	347	18	3	135
default	currencyservice-5574946d56-s4l4f	●	1/1	0	Running	0	0	0	0	0
default	currencyservice-5574946d56-tzznf	●	1/1	0	Running	0	0	0	0	0
default	currencyservice-5574946d56-vc62c	●	1/1	0	Running	5	44	5	1	17
default	emailservice-5f757cdb8c-jdxj2	●	1/1	1	Running	2	10	n/a	n/a	n/a
default	frontend-dc949f759-xp9bz	●	0/1	2	OOMKilled	489	933	97	32	182
default	loadgenerator-6db5967845-8nlq6	●	1/1	0	Running	325	809	n/a	n/a	n/a

Anexo 28. Resultado de microservicios con HPA activado.



Tú

autoscaling/v1 para memoria ram



ChatGPT

En `autoscaling/v1`, solo puedes escalar según la CPU. Si deseas escalar según la memoria RAM, necesitarás considerar otras opciones, como crear un script personalizado que monitoree la memoria RAM de tus pods y ajuste el número de réplicas en función de eso.

Sin embargo, si realmente necesitas escalar según la memoria RAM y estás dispuesto a utilizar `autoscaling/v2beta2`, aquí tienes un ejemplo de cómo podrías configurar un HPA para escalar según la memoria RAM:

Anexo 29. Respuesta desactualizada de ChatGPT referente a versiones de autoscaling

Horizontalpodautoscalers(default)[2]						
NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	
currencyservice-autoscaler	Deployment/currencyservice	78%/50%	1	10	9	
frontend-autoscaler	Deployment/frontend	41%/50%, 734259882666m/1000Mi	1	10	3	

Anexo 30. Parámetros de HPA siendo utilizados en CPU y RAM.

NAME	PF	READY	RESTARTS	STATUS	CPU	MEM	%CPU/R	%CPU/L	%MEM/R	%MEM/L
adservice-796df94b99-sbr77	●	1/1	1	Running	51	268	n/a	n/a	n/a	n/a
cartservice-7cf97f4966-ncq6g	●	1/1	1	Running	61	125	n/a	n/a	n/a	n/a
checkoutservice-6f87cb4b4d-7st7v	●	1/1	1	Running	6	15	n/a	n/a	n/a	n/a
currencyservice-5574946d56-5v2ch	●	1/1	0	Running	12	15	12	2	6	3
currencyservice-5574946d56-8ptsm	●	1/1	0	Running	131	55	131	26	21	10
currencyservice-5574946d56-k8nw2	●	1/1	0	Running	14	15	14	2	6	3
currencyservice-5574946d56-k9vsq	●	1/1	0	Running	5	25	5	1	9	4
currencyservice-5574946d56-kfb9w	●	1/1	0	Running	330	91	330	66	35	17
currencyservice-5574946d56-n7t5j	●	1/1	0	Running	14	15	14	2	6	3
currencyservice-5574946d56-pc44q	●	1/1	1	Running	1	16	1	0	6	3
currencyservice-5574946d56-qf8lg	●	1/1	0	Running	1	15	1	0	6	3
currencyservice-5574946d56-wvcrx	●	1/1	0	Running	1	16	1	0	6	3
emailservice-5f757cdb8c-jdxj2	●	1/1	1	Running	2	8	n/a	n/a	n/a	n/a
frontend-dc949f759-7rbpr	●	1/1	2	Running	201	976	40	13	190	65
frontend-dc949f759-gj89f	●	1/1	0	Running	126	130	25	8	25	8
frontend-dc949f759-qzdzk	●	1/1	0	Running	215	993	43	14	194	66
loadgenerator-6db5967845-77nm6	●	1/1	0	Running	195	884	n/a	n/a	n/a	n/a
loadgenerator-6db5967845-cwphd	●	1/1	0	Running	201	867	n/a	n/a	n/a	n/a

Anexo 31. Uso de CPU y memoria de los contenedores durante el ejercicio de generación de carga con HPA activados.

14	spec:	29	spec:
		30+	serviceAccountName: default
		31+	securityContext:
		32+	fsGroup: 1000
		33+	runAsGroup: 1000
		34+	runAsNonRoot: true
		35+	runAsUser: 1000
15	containers:	36	containers:
16-	- name: frontend	37+	- name: server
17-	image: fsaezv/frontend:latest	38+	securityContext:
		39+	allowPrivilegeEscalation: false
		40+	capabilities:
		41+	drop:
		42+	- ALL
		43+	privileged: false
		44+	readOnlyRootFilesystem: true
		45+	image: frontend
18-	ports:	46+	ports:
19-	- containerPort: 8080	47+	- containerPort: 8080
		48+	readinessProbe:
		49+	initialDelaySeconds: 10
		50+	httpGet:
		51+	path: "/_healthz"
		52+	port: 8080
		53+	httpHeaders:
		54+	- name: "Cookie"
		55+	value: "shop_session-id=x-readiness-probe"
		56+	livenessProbe:
		57+	initialDelaySeconds: 10
		58+	httpGet:
		59+	path: "/_healthz"
		60+	port: 8080
		61+	httpHeaders:
		62+	- name: "Cookie"
		63+	value: "shop_session-id=x-liveness-probe"

Anexo 32. Diferencias entre configuración provista por ChatGPT (izquierda) y realizada por proveedor (derecha).

Controls	1
Passed	0
Failed	1
Action Required	0

Failed resources by severity:

Critical	0
High	0
Medium	13
Low	0

Severity	Control name	Failed resources	All Resources	Compliance score
Medium	Missing network policy	13	33	61%
	Resource Summary	13	33	60.61%

Anexo 35. Resultado de falta de políticas de red en clúster.

Tú
la conexión es desde frontend hacia los demás, no al revés

ChatGPT
Gracias por la corrección. En ese caso, la Network Policy debería permitir el tráfico de salida desde el microservicio de frontend hacia los demás servicios. Aquí tienes la Network Policy ajustada:

```

yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-egress-from-frontend
spec:
  podSelector:
    matchLabels:
      app: frontend
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: productcatalogservice
        - podSelector:
            matchLabels:
              app: currencyservice

```

Anexo 36. Configuración nueva entregada por ChatGPT referente a las políticas de red.

Failed resources by severity:

Critical	0
High	0
Medium	12
Low	0

Severity	Control name	Failed resources	All Resources	Compliance score
Medium	Missing network policy	12	34	65%
	Resource Summary	12	34	64.71%

Anexo 37. Mitigación de microservicio frontend.

Workload

Control name	Resources	View details
Host PID/IPC privileges	0	\$ kubescape scan control C-0038 -v
HostNetwork access	0	\$ kubescape scan control C-0041 -v
HostPath mount	0	\$ kubescape scan control C-0048 -v
Non-root containers	10	\$ kubescape scan control C-0013 -v
Privileged container	0	\$ kubescape scan control C-0057 -v

Anexo 38. Reporte respecto al Workload con kubescape.

```
#####
ApiVersion: apps/v1
Kind: Deployment
Name: frontend
Namespace: default
Controls: 1 (Failed: 1, action required: 0)
```

Resources	
Severity	: Medium
Control Name	: Non-root containers
Docs	: https://hub.armosec.io/docs/c-0013
Assisted Remediation	: spec.template.spec.containers[0].securityContext.runAsNonRoot=true spec.template.spec.containers[0].securityContext.runAsGroup=1000

Anexo 39. Reporte respecto a frontend en Workload con kubescape.

Workload

Control name	Resources	View details
Host PID/IPC privileges	0	\$ kubescape scan control C-0038 -v
HostNetwork access	0	\$ kubescape scan control C-0041 -v
HostPath mount	0	\$ kubescape scan control C-0048 -v
Non-root containers	9	\$ kubescape scan control C-0013 -v
Privileged container	0	\$ kubescape scan control C-0057 -v

Anexo 40. Reporte respecto al Workload con kubescape con mitigación incluida.

```
1 services:
2   frontend:
3     build: ./src/frontend
4     ports:
5       - "8100:8080"
6     environment:
7       - DISABLE_PROFILER=1
8       - PRODUCT_CATALOG_SERVICE_ADDR=productcatalogservice:3550
9       - CURRENCY_SERVICE_ADDR=currencyservice:7000
10      - CART_SERVICE_ADDR=cartservice:7070
11      - RECOMMENDATION_SERVICE_ADDR=recommendationservice:8080
12      - SHIPPING_SERVICE_ADDR=shippingservice:50051
13      - CHECKOUT_SERVICE_ADDR=checkoutservice:5050
14      - AD_SERVICE_ADDR=adservice:9555
15   adservice:
16     build: ./src/adservice
17     environment:
18       - DISABLE_PROFILER=1
19   cartservice:
20     build: ./src/cartservice/src
21     environment:
22       - DISABLE_PROFILER=1
23   checkoutservice:
24     build: ./src/checkoutservice
25     environment:
26       - DISABLE_PROFILER=1
27       - PRODUCT_CATALOG_SERVICE_ADDR=productcatalogservice:3550
```

Anexo 41. Parte del Docker Compose resultante.

Your order is complete!

We've sent you a confirmation email.

Confirmation #	6405f2f9-fe8e-11ee-b462-0242ac14000b
Tracking #	NS-44897-229164704
Total Paid	\$28.98

Continue Shopping

Anexo 42. Sitio web funcionando correctamente.

```
WARN Service "adservice" won't be created because 'ports' is not specified
WARN Service "cartservice" won't be created because 'ports' is not specified
WARN Service "checkoutservice" won't be created because 'ports' is not specified
WARN Service "currencyservice" won't be created because 'ports' is not specified
WARN Service "emailservice" won't be created because 'ports' is not specified
WARN Service "paymentservice" won't be created because 'ports' is not specified
WARN Service "productcatalogservice" won't be created because 'ports' is not specified
WARN Service "recommendationservice" won't be created because 'ports' is not specified
WARN Service "shippingservice" won't be created because 'ports' is not specified
INFO Kubernetes file "frontend-service.yaml" created
INFO Kubernetes file "adservice-deployment.yaml" created
INFO Kubernetes file "cartservice-deployment.yaml" created
INFO Kubernetes file "checkoutservice-deployment.yaml" created
INFO Kubernetes file "currencyservice-deployment.yaml" created
INFO Kubernetes file "emailservice-deployment.yaml" created
INFO Kubernetes file "frontend-deployment.yaml" created
INFO Kubernetes file "paymentservice-deployment.yaml" created
INFO Kubernetes file "productcatalogservice-deployment.yaml" created
INFO Kubernetes file "recommendationservice-deployment.yaml" created
INFO Kubernetes file "shippingservice-deployment.yaml" created
```

Anexo 43. Creación de archivos de configuración con kompose.

Después de revisar la información proporcionada, podemos identificar las siguientes sub-tareas a partir de la salida del Dockerfile:

1. Identificar y documentar los puertos utilizados por la aplicación.
Puntaje: 1
2. Crear un archivo de configuración de Kubernetes para el Deployment del microservicio.
 - 2.1 Definir las especificaciones del contenedor.
 - 2.2 Establecer el puerto de exposición del contenedor.Puntaje: 2
3. Configurar un Service de Kubernetes para permitir la comunicación con el microservicio.
 - 3.1 Definir el Service con el mismo puerto expuesto en el contenedor.
 - 3.2 Establecer el tipo de Service (ClusterIP, NodePort, LoadBalancer).Puntaje: 2

Para la siguiente tarea, por favor investiga cómo configurar un Ingress en Kubernetes para permitir el acceso externo a los microservicios desplegados. Documenta los pasos necesarios y cualquier consideración relevante para su implementación.

Anexo 44. Salida de aplicación desarrollada con integración OpenAI.



You may start with:

Por supuesto. A continuación se detalla la tarea específica con la instrucción para ejecutarla de manera clara y concisa:

****Tarea: Aplicar una configuración YAML para desplegar un microservicio en Kubernetes****

Instrucción para el devops engineer junior:

Ejecuta el siguiente comando para aplicar la configuración YAML que desplegará un microservicio en Kubernetes:

```
``bash
kubectl apply -f nombre-del-archivo.yaml
``
```

Asegúrate de que el archivo YAML contenga la especificación necesaria para el despliegue del microservicio. Una vez ejecutado el comando, verifica que el despliegue se haya realizado correctamente utilizando 'kubectl get pods' y 'kubectl get services'.

> |

Anexo 45. Salida de aplicación desarrollada con integración OpenAI.